

Classification & Decision Trees

Oct. 21, 2025 - Lecture 6

Outline

- Classification
 - Logistic Regression, SVM, KNN
 - Classification Metrics
- Decision trees
 - Classification trees, regression trees
- Ensemble learning
 - Bagging
 - Random forest, out of bag sample
 - Boosting
 - AdaBoost, Gradient Boosting, XGBoost
 - Feature Importance

Classification

Classification is a supervised machine learning method where a model tries to predict the **correct label** for given input data.

In **Binary Classification**, the model has to distinguish between 2 classes (e.g. labelling an email spam or not-spam).

In **Multi-Class Classification**, the model has to distinguish more than 2 classes (e.g. image classification).

Classification vs. Regression

Aspect	Classification	Regression
Goal	Predict categories or labels	Predict continuous numeric values
Output	Discrete values (e.g. spam / not spam, dog / cat) or Continuous values for likelihood that prediction belongs to a certain class (85% dog / 15% cat)	Continuous values (e.g. price, temperature, age)
Evaluation Metrics	Accuracy, Precision, Recall, F1-score	Mean Squared Error (MSE), Mean Absolute Error (MAE), R^2
Example Algorithms	Logistic Regression, SVM, KNN, Decision Trees	Linear Regression, Ridge, Lasso, Decision Trees

Logistic Regression

Logistic Regression is a classification model that predicts the probability that a specific outcome occurs. It outputs a value between **0 and 1** which represents the **likelihood** of something belonging to a certain class.

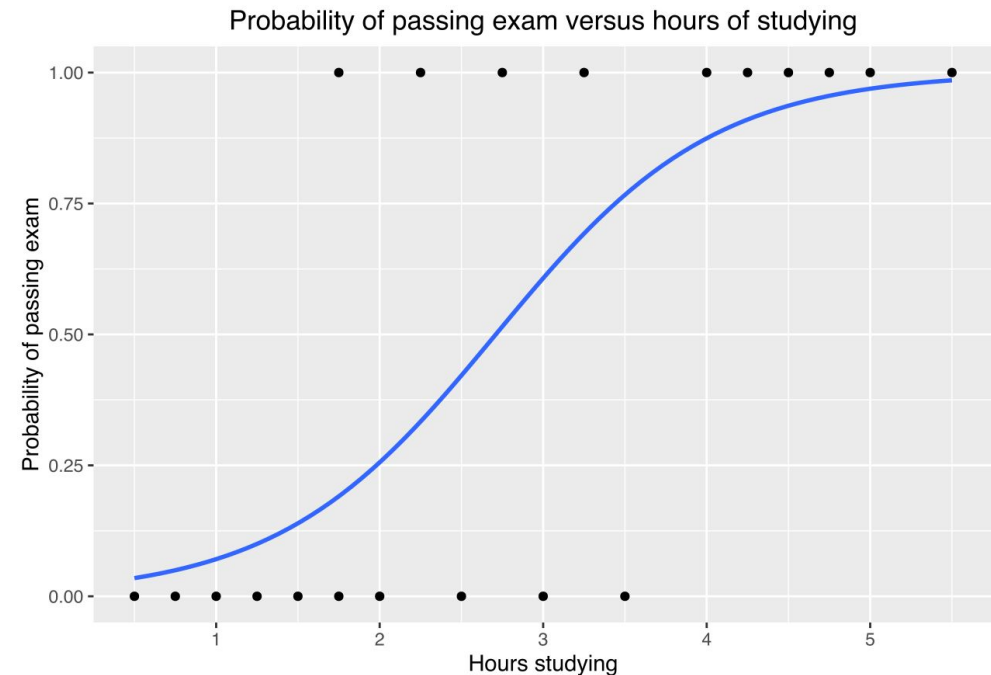
- For example, if we're trying to predict whether an email is spam or not spam, logistic regression might give us a value like 0.85, meaning there's an 85% chance the email is spam.

It's one of the simplest and most common models used for binary classification problems.

Logistic Regression: Sigmoid Function

Logistic Regression is similar to linear regression, but it uses the **sigmoid function** to squeeze predictions into the 0 - 1 range. Note this is for binary classification only.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Logistic Regression

Since **Logistic Regression** outputs probabilities, we need a **threshold** - a **cutoff value** we use to decide which class a prediction belongs to based on its predicted probability or score.

- Output $\geq$ threshold $\rightarrow$ predict positive class (1)
- Output $<$ threshold $\rightarrow$ predict negative class (0)

Logistic Regression: Multi-Class

We can extend logistic regression to use it for multi-class classification.

For multi-class classification problems, we use the **softmax function** instead of the sigmoid function. **Softmax** takes a list of scores (one for each class) and turns them into probabilities:

$$P(y_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

All probabilities sum to 1, and we predict the class with the highest probability.

Logistic Regression

Pros:

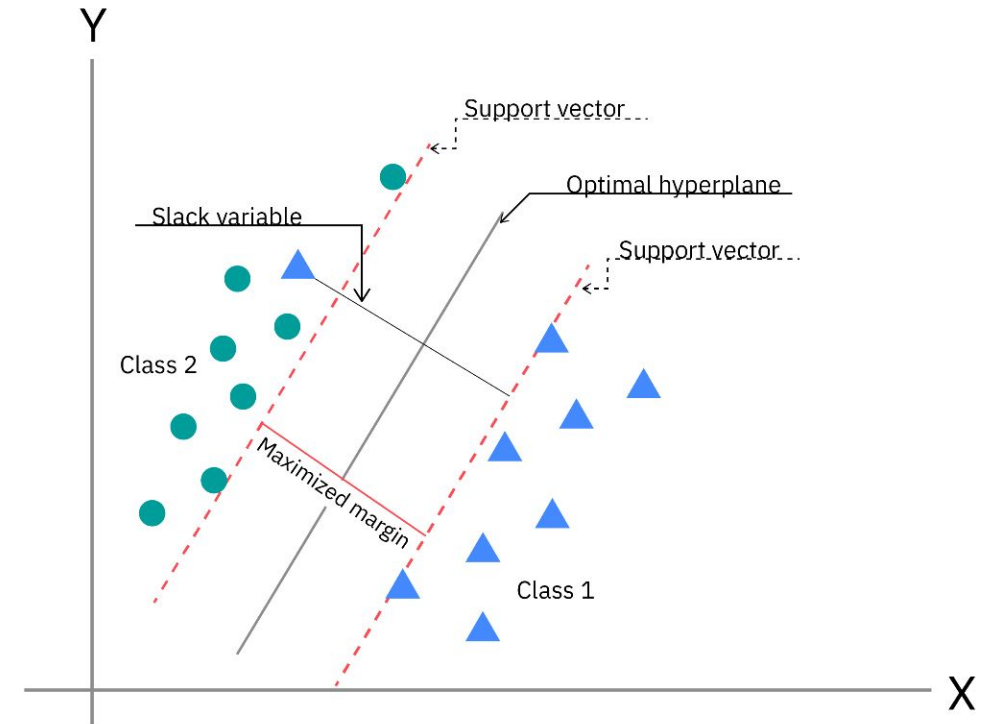
- Interpretable - coefficients show how each feature affects the outcome.
- Works well with linearly separable data.

Cons:

- Can struggle with complex, nonlinear data.

SVM (Support Vector Machine)

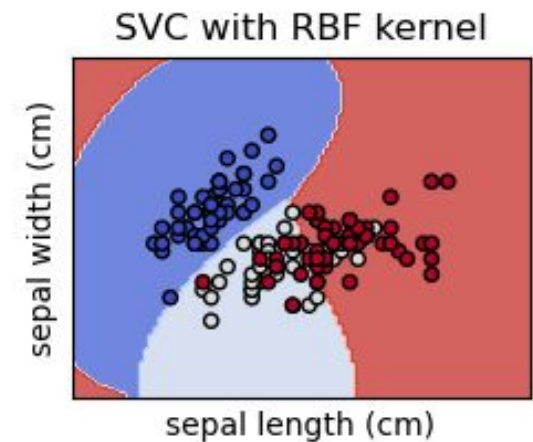
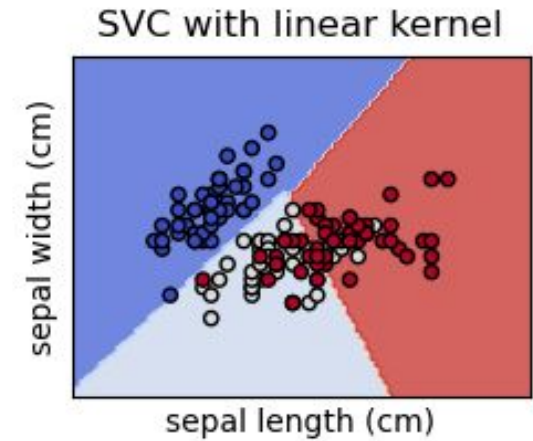
SVM (support vector machine) is a supervised machine learning algorithm that can be used for classification. It tries to find the best boundary (called a **decision boundary** or hyperplane) that **separates different classes** of data.



SVM (Support Vector Machine)

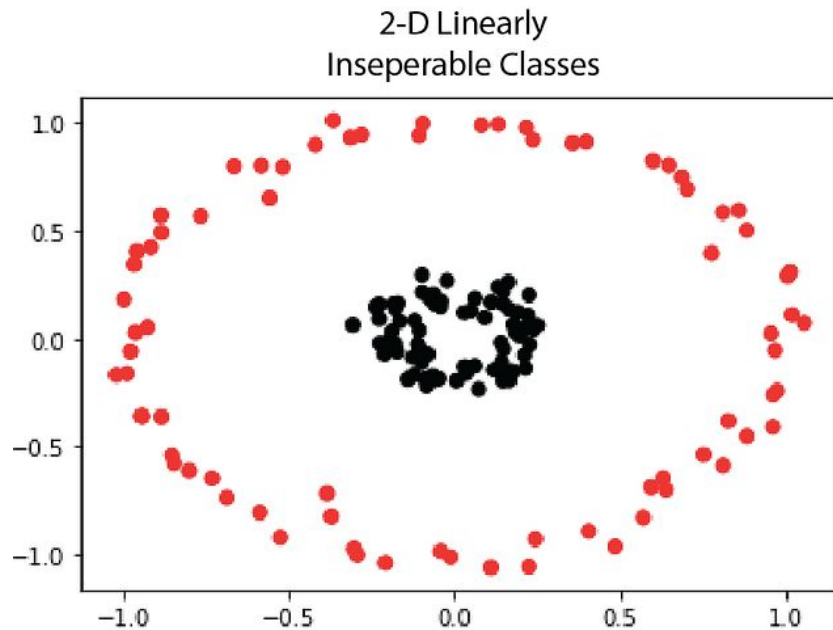
There are different types of **SVMs**:

- **Linear SVMs** are used with linearly separable data.
- **NonLinear SVMs** are used when data is not linearly separable (which is common in a lot of real world data). To make the data linearly separable, SVMs use **kernel functions**.



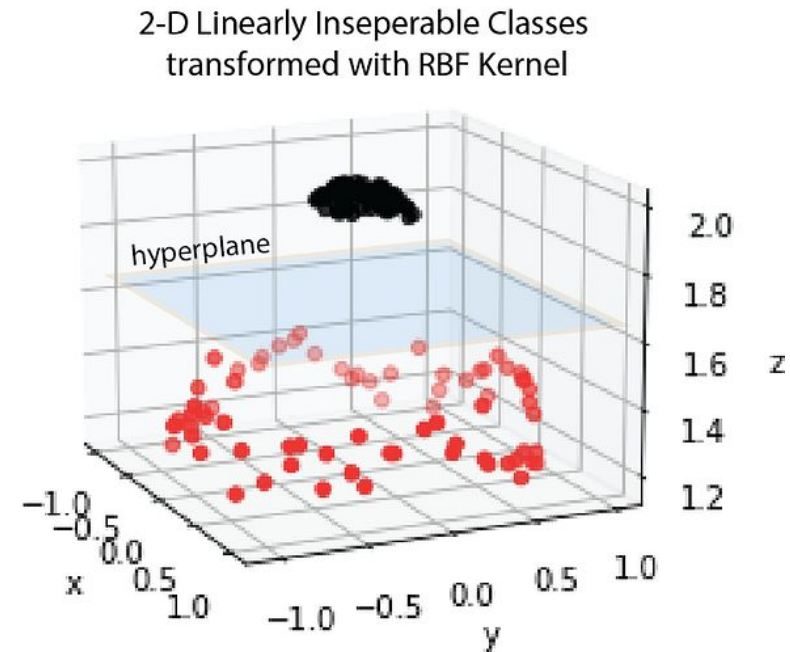
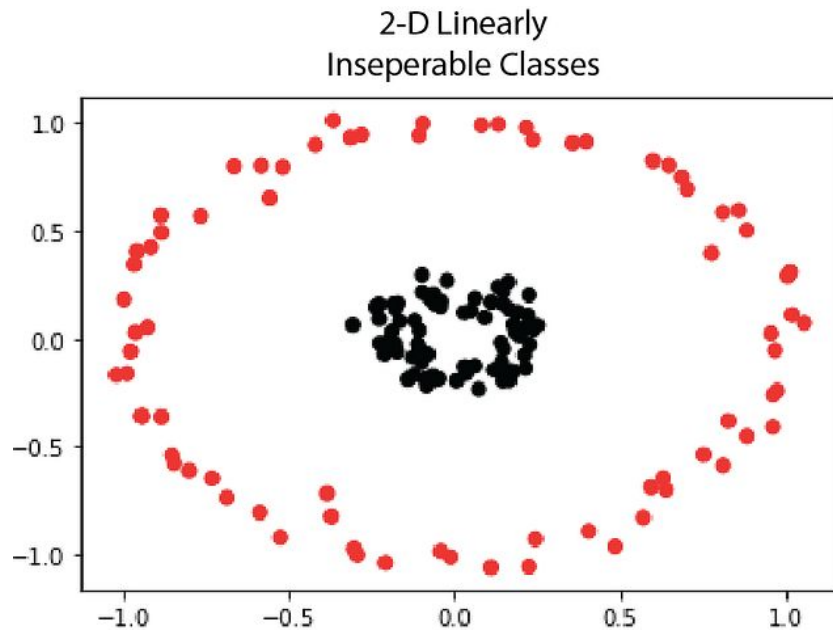
SVM Kernels

- A **kernel** in SVMs is a function that measures the similarity between two data points but in a way that assumes they've been mapped into a higher-dimensional feature space.



SVM Kernels

- A **kernel** in SVMs is a function that measures the similarity between two data points but in a way that assumes they've been mapped into a higher-dimensional feature space.



SVMs - Scaling

Scaling before using SVMs is very important!

- SVM tries to maximize the **distance** between the separating plane and the support vectors (the data points that lie closest to the decision boundary).
- If one feature has very large values, it will dominate the other features when calculating the distance.
- If you **rescale all features** they all have the same influence on the distance metric.

SVM

Pros:

- Works well for both linear and nonlinear problems.

Cons:

- Choosing the right kernel and parameters can be difficult.
- Harder to interpret than logistic regression.

KNN (K-Nearest Neighbors)

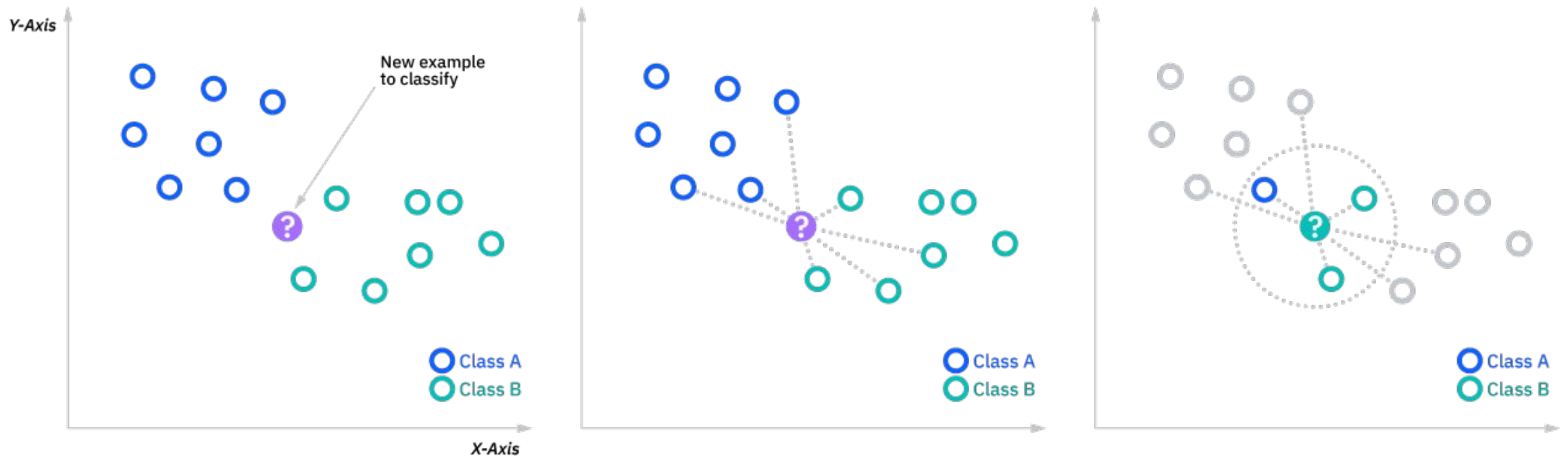
KNN (K-Nearest Neighbors) is one of the simplest classification algorithms. It looks at the closest K data points (neighbors) to a new example and decides its class based on majority vote.

1. Choose a value for K (for example, $K = 5$).
2. When a new data point comes in, find its K nearest neighbors in the training set (usually using distance).
3. Count how many neighbors belong to each class.
4. The new point is classified as the class with the most votes among its neighbors.

Just as with SVM, KNN **requires scaling** beforehand.

KNN: Visual Example

In this case, the new point will be classified as belonging to Class B (out of the 3 nearest neighbors, $\frac{2}{3}$ are from Class B).



KNN

Pros:

- Very easy to understand and implement.

Cons:

- Performance depends heavily on choice of K, distance metric, and how much data you have.
- Sensitive to irrelevant or highly correlated features.

Evaluation Metrics for Classification

Metric	Formula	Explanation
Accuracy	$\text{Accuracy} = \frac{\# \text{ points correctly classified}}{\text{total } \# \text{ of points}}$	Overall proportion of correct predictions.
Precision	$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$	Measures how many of the predicted positives are truly positive.
Recall (Sensitivity)	$\text{Sensitivity} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$	The proportion of positives that are correctly labelled.
Specificity	$\text{Specificity} = \frac{\text{True Negatives}}{\text{True Negatives} + \text{False Positives}}$	The proportion of negatives that are correctly labelled.
ROC-AUC	<i>Area under the ROC curve</i>	Measures how well the model can distinguish between classes across all thresholds — higher is better.

True/False Positives/Negatives

Confusion Matrix

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Evaluation Metrics: Accuracy

Note that accuracy can be **misleading** if the classes are **imbalanced**! Typically, we compare accuracy to the **baseline accuracy**: the accuracy when guessing at random.

- Suppose we have a data set where 990 cases are not fraud and 10 cases are fraud.
- If our model **only predicts not fraud, it achieves 99% accuracy!**
- However, this model is useless because it can't actually detect fraud.

In these cases, we can use **balanced accuracy** (gives equal weight to the performance of both classes) instead, or we can choose to maximize a metric **other than accuracy**.

Evaluation Metrics: ROC

An ROC curve is a plot of the **true positive rate (TPR)** against the **false positive rate (FPR)** for different **thresholds** (ranging from 0-100%).

$$TPR = \frac{TP}{TP + FN} \qquad FPR = \frac{FP}{FP + TN}$$

TPR: Out of all the actual positives (class 1), how many did we get right?

FPR: Out of all the actual negatives (class 2), how many did we get wrong?

A "perfect model" will have a **TPR of 1.0** and a **FPR of 0.0** at **some threshold**.

Example of Setting Thresholds

Your model outputs probabilities (how likely each patient **has** the disease). You can set different decision **thresholds** for classification. Higher thresholds mean you're more cautious about calling someone "positive". Ex. The model is 60% confident a patient **has** a disease. If we have a threshold of 50%, then they will be classified as having the disease; conversely, if we set the threshold to 85%, then they will be classified as **not having** the disease.

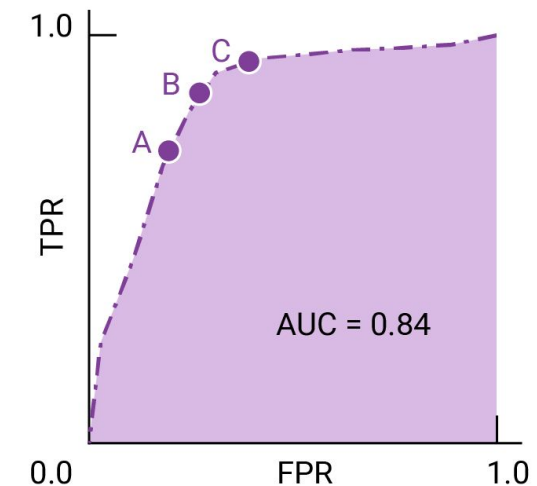
Thresholds determine your classifications!

ROC: Finding the Best Threshold

We can use the ROC curve to find the **best threshold** for our model. The points on a ROC curve closest to the top left corner of the plot (0,1) represent a range of the best-performing **thresholds** for the given model. In this example these points are labelled A,B,C.

This is because those points will maximize the TPR while minimizing the FPR.

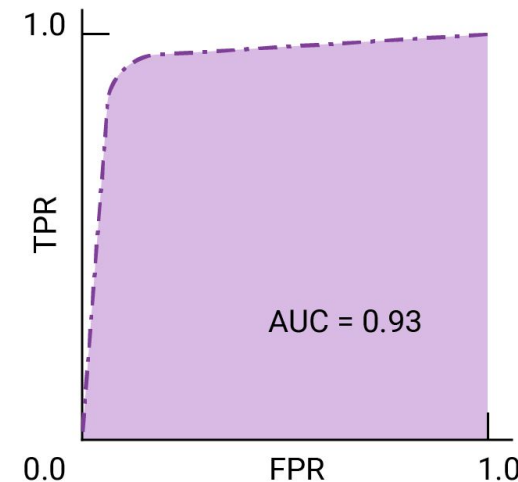
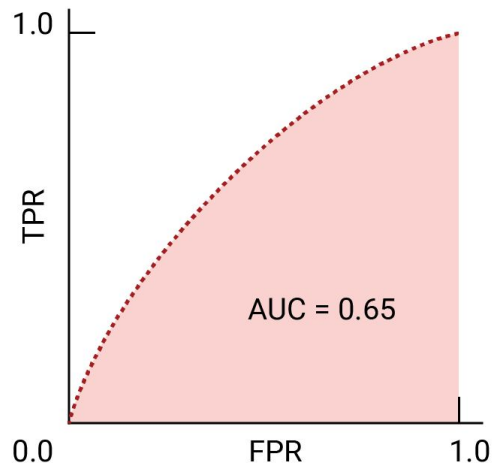
For an interactive example see this [Google Crash Course page](#).



Some candidates for best thresholds.

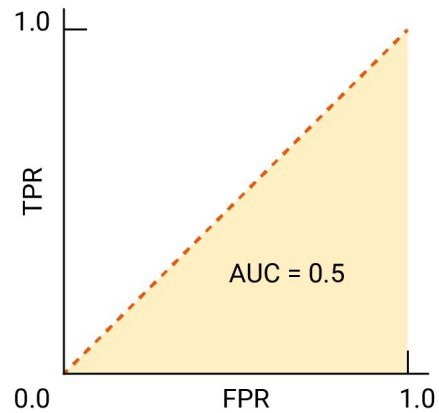
Evaluation Metrics: AUC

AUC (Area Under the ROC Curve) is an evaluation metric that can be used to **compare the performance** of different models. The model with **greater AUC** is generally the better one.

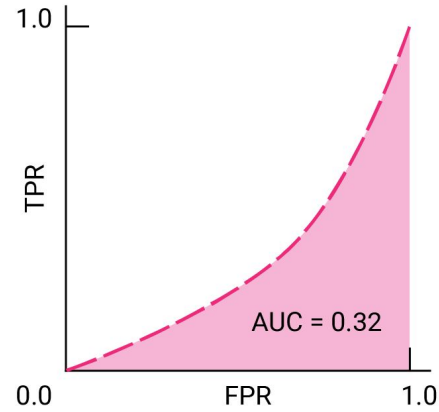


An ideal ROC curve will hug the top left corner, so the **larger area under the (ROC) curve the AUC the better the classifier.**

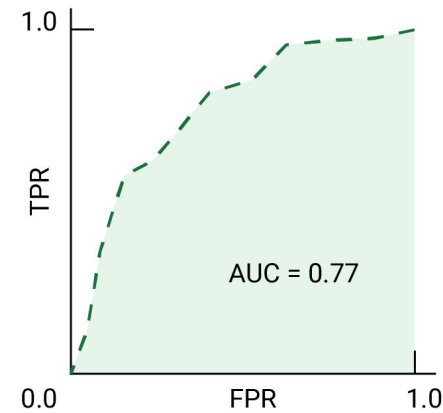
ROC Examples



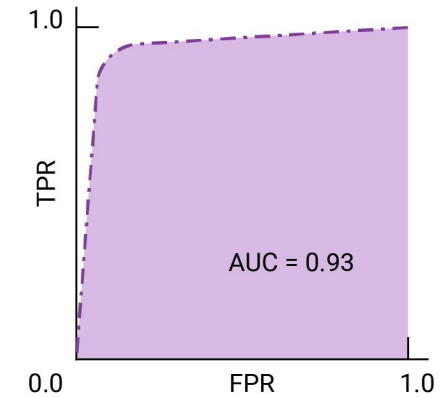
ROC curve of completely random guesses.



ROC curve of a model that is worse than random chance.



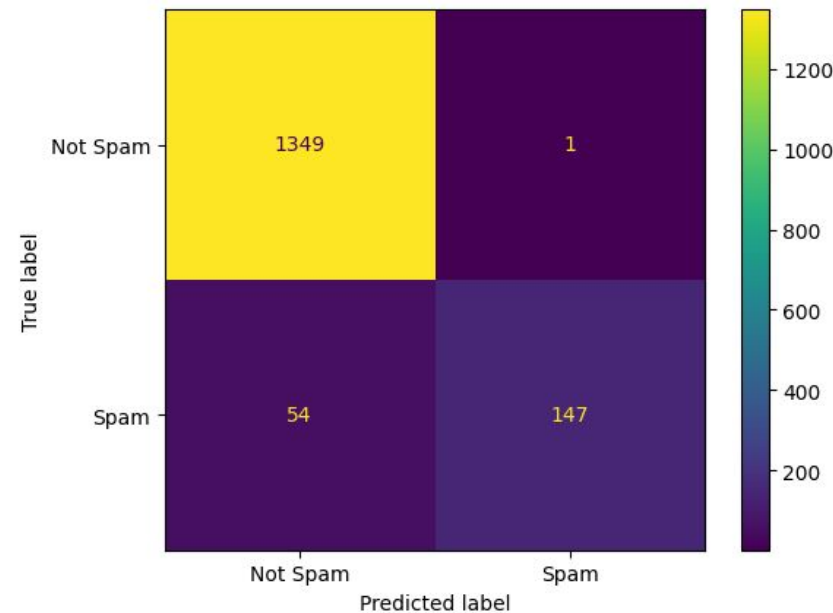
ROC curve of a better performing model.



ROC curve of an even better performing model.

Evaluation Metrics: Confusion Matrix

A **confusion matrix** can be helpful to summarize where our model made mistakes in its predictions.

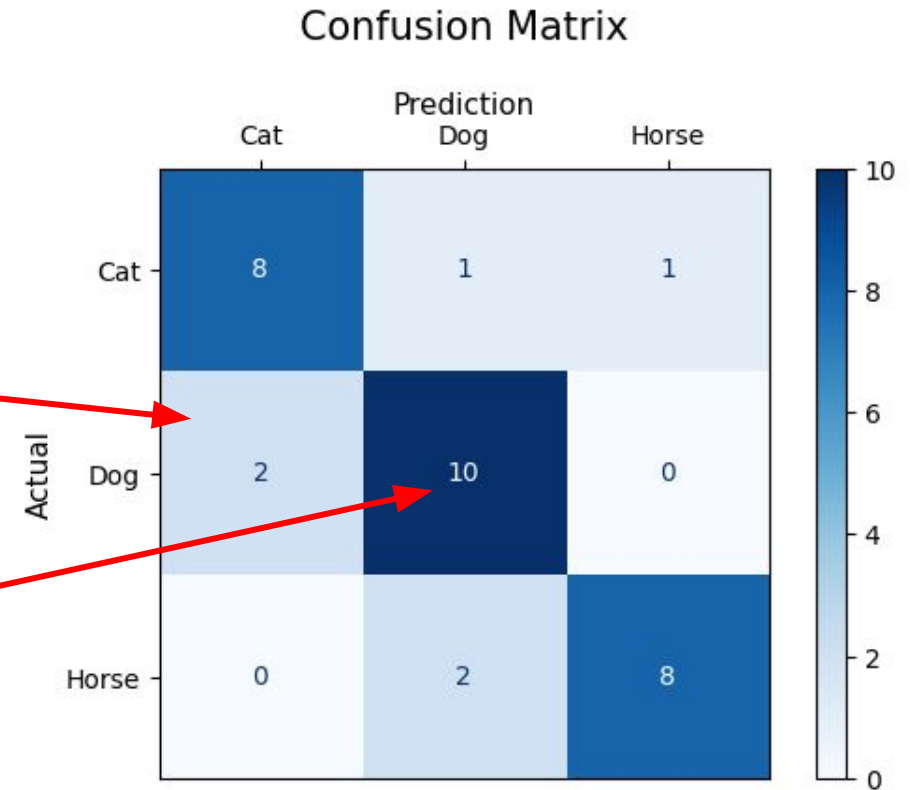


Evaluation Metrics: Confusion Matrix

Confusion matrices can be used for multiple classes as well.

e.g. This model incorrectly classified 2 dogs as cats.

e.g. This model correctly classified 10 dogs.



Summary

- **Classification** is a supervised machine learning problem where we try to predict a qualitative output (e.g. labels, classes, categories).
 - This is different from regression, where the output is quantitative.
- There are several **evaluation metrics** for classification, and which to use depends on what you want to maximize.
 - Some metrics are misleading when **classes are imbalanced!**
- Many models can be used for classification, including:
 - **Logistic regression** - models the probability that a data point belongs to each class using a linear decision boundary.
 - **SVM** - finds the boundary that best separates classes by maximizing the margin between them.
 - **KNN** - makes a prediction based on the classes of the K-nearest neighbors to the new data point.

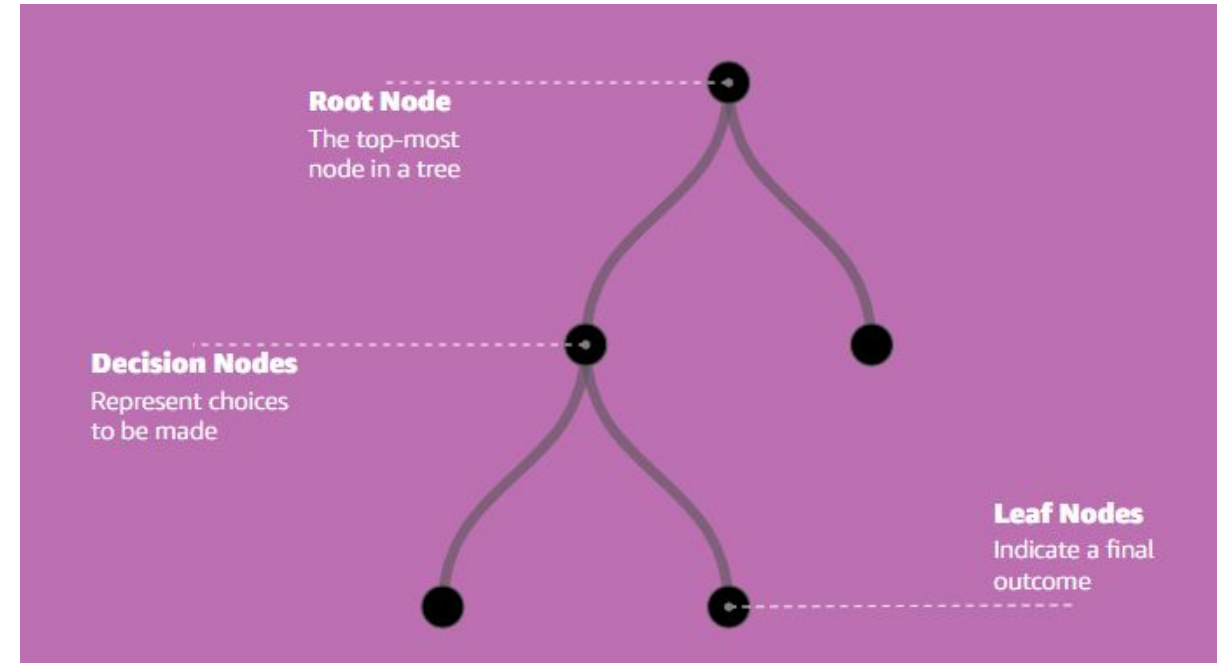
Decision Trees

Decision Trees (DT)

A **Decision Tree (DT)** consists of a series of sequential decisions (decision nodes) on some dataset's features.

The tree is navigated via **if-then rules**, which split each decision node into **two or more subnodes**.

Leaf nodes, also known as **terminal nodes**, represent **prediction outputs** for the model.



Decision Trees (DT) Continued

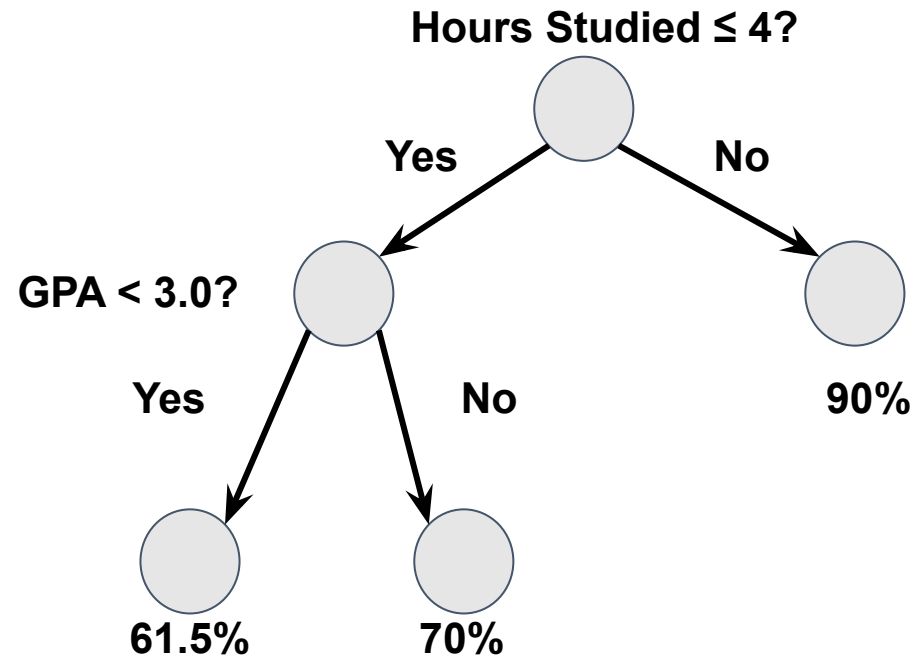
Decision trees can be used for both regression and classification problems.

- We create **splits** based on some feature in the dataset (e.g. if height > 2.4m then...., else)
- We choose our splits based on which one has the **highest information gain**.
- The deeper the decision tree, the **more complex** the decision rules.

Regression Tree: Visual Example

Example: Suppose we want to predict a student's final grade in DS3000 using their GPA and hours studied per week. Given the data on the left, we construct a **regression tree**.

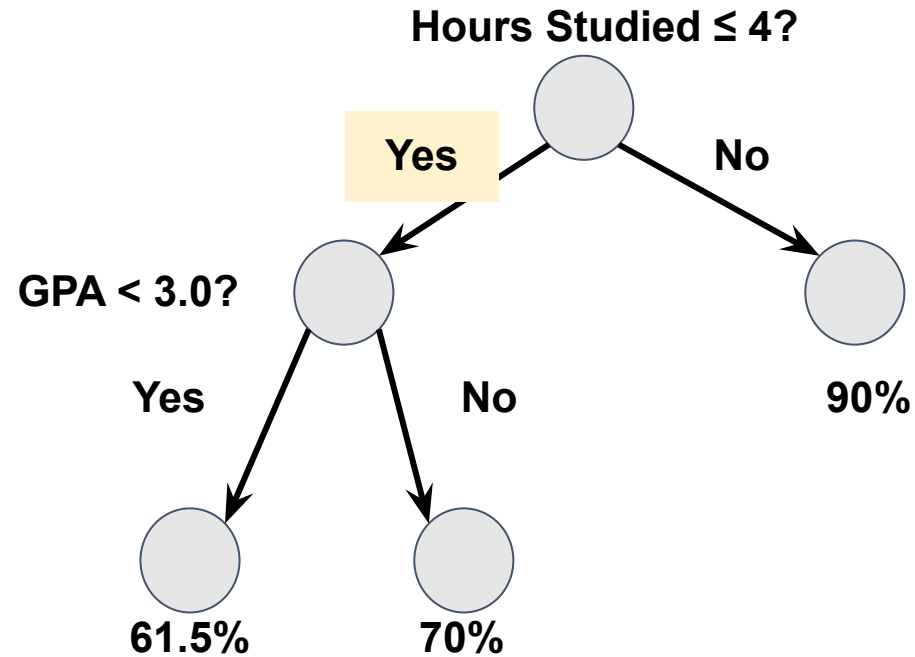
GPA	Hours Studied	Final Grade
2.0	3	60%
3.0	4	70%
3.5	6	80%
2.8	3	63%
4.0	10	100%



Regression Tree: Visual Example

Let's traverse down the leftmost side of the tree.

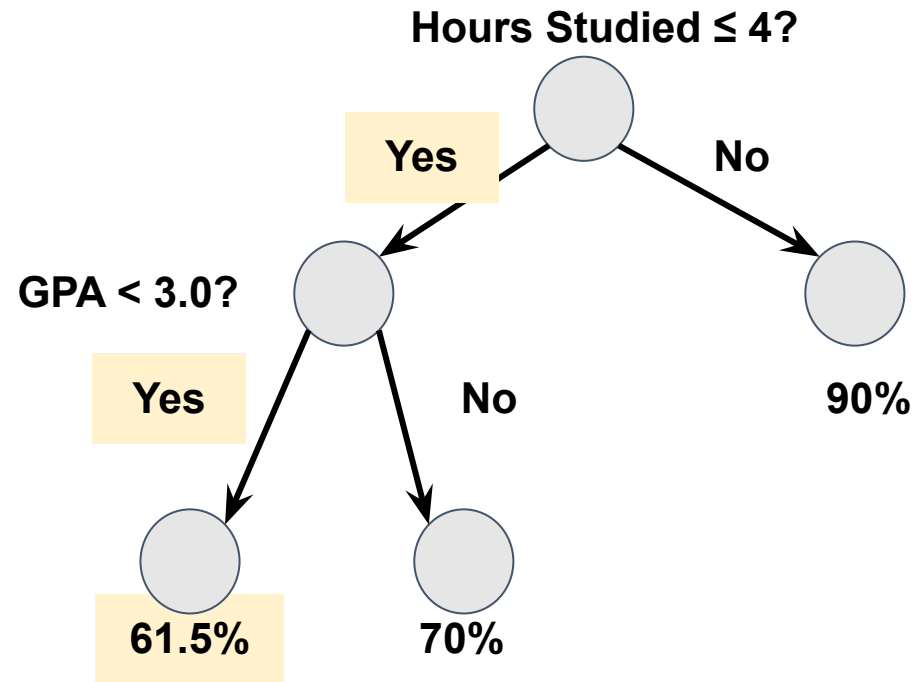
GPA	Hours Studied	Final Grade
2.0	3	60%
3.0	4	70%
3.5	6	80%
2.8	3	63%
4.0	10	100%



Regression Tree: Visual Example

Our final prediction is the average of the samples in the leaf node. In this case the average of 60% and 63% is 61.5%.

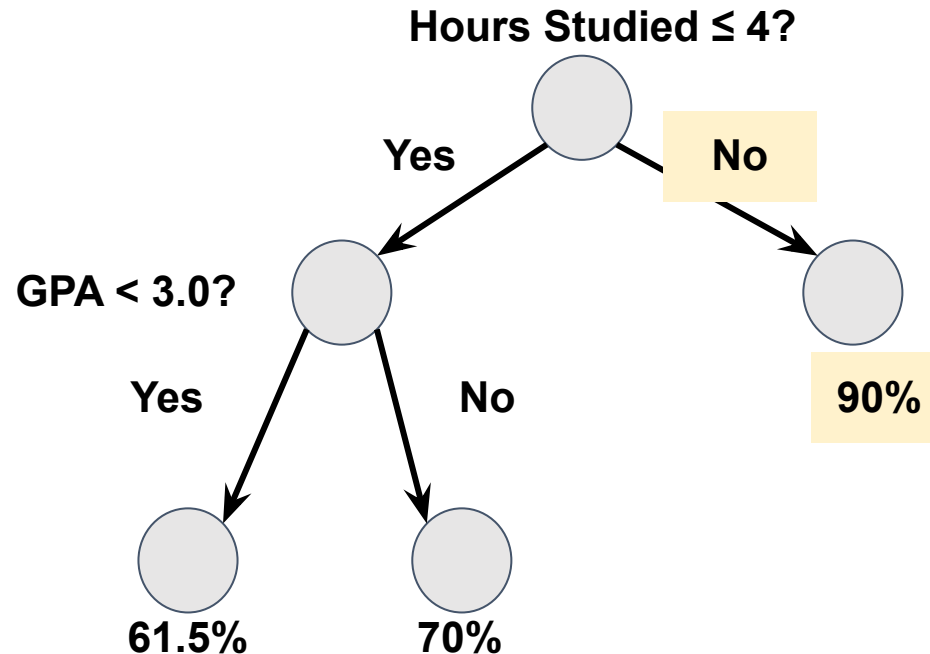
GPA	Hours Studied	Final Grade
2.0	3	60%
3.0	4	70%
3.5	6	80%
2.8	3	63%
4.0	10	100%



Regression Tree: Visual Example

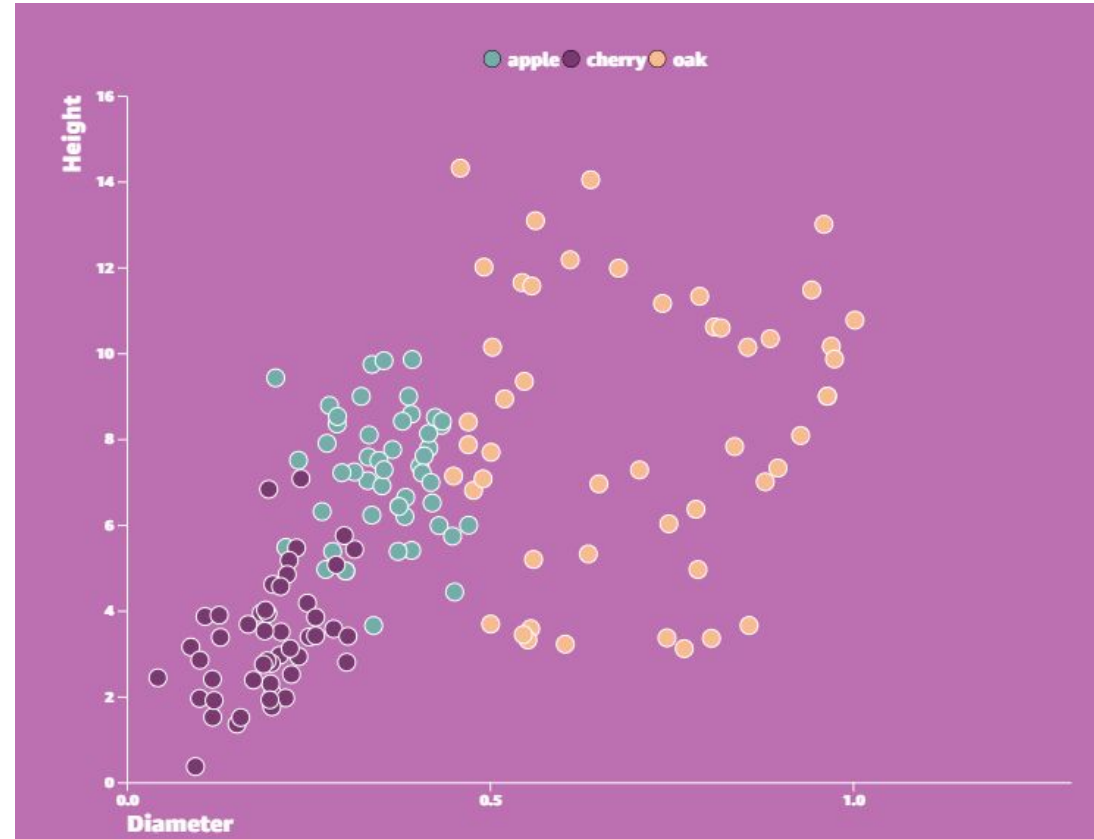
Similarly, for the leaf node on the right, the predicted grade is 90%, which is the average of the 2 students' grades who have 4+ hours studied.

GPA	Hours Studied	Final Grade
2.0	3	60%
3.0	4	70%
3.5	6	80%
2.8	3	63%
4.0	10	100%

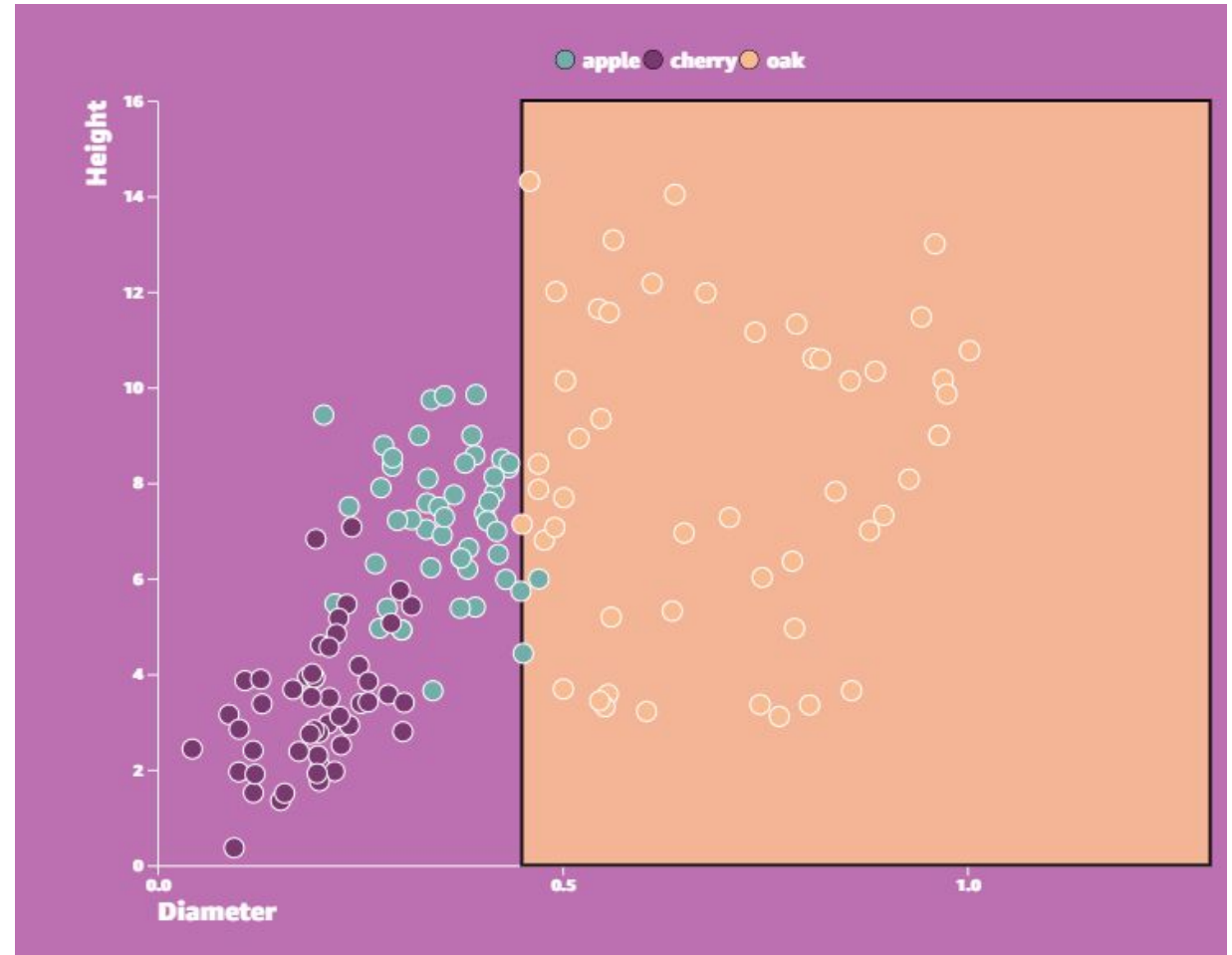
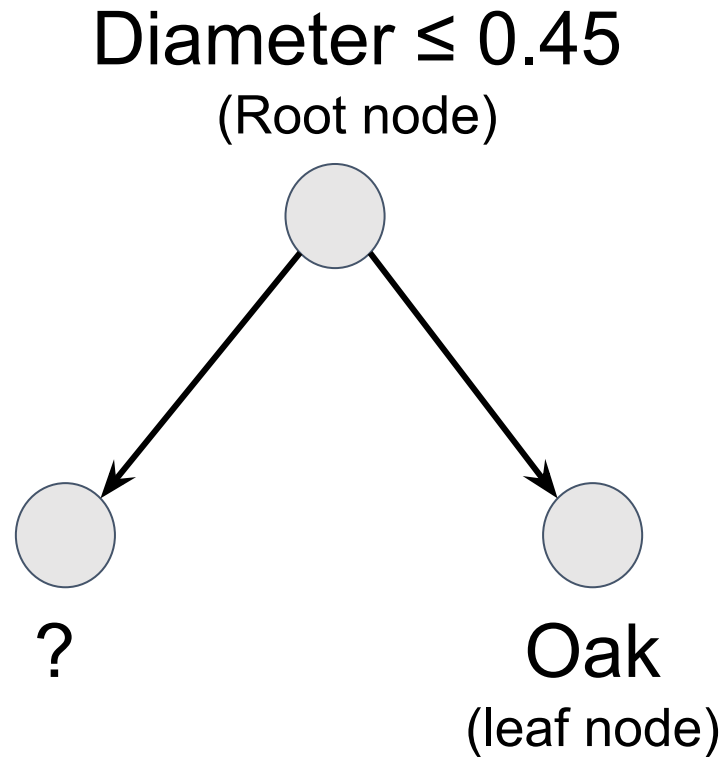


Classification Tree: Visual Example

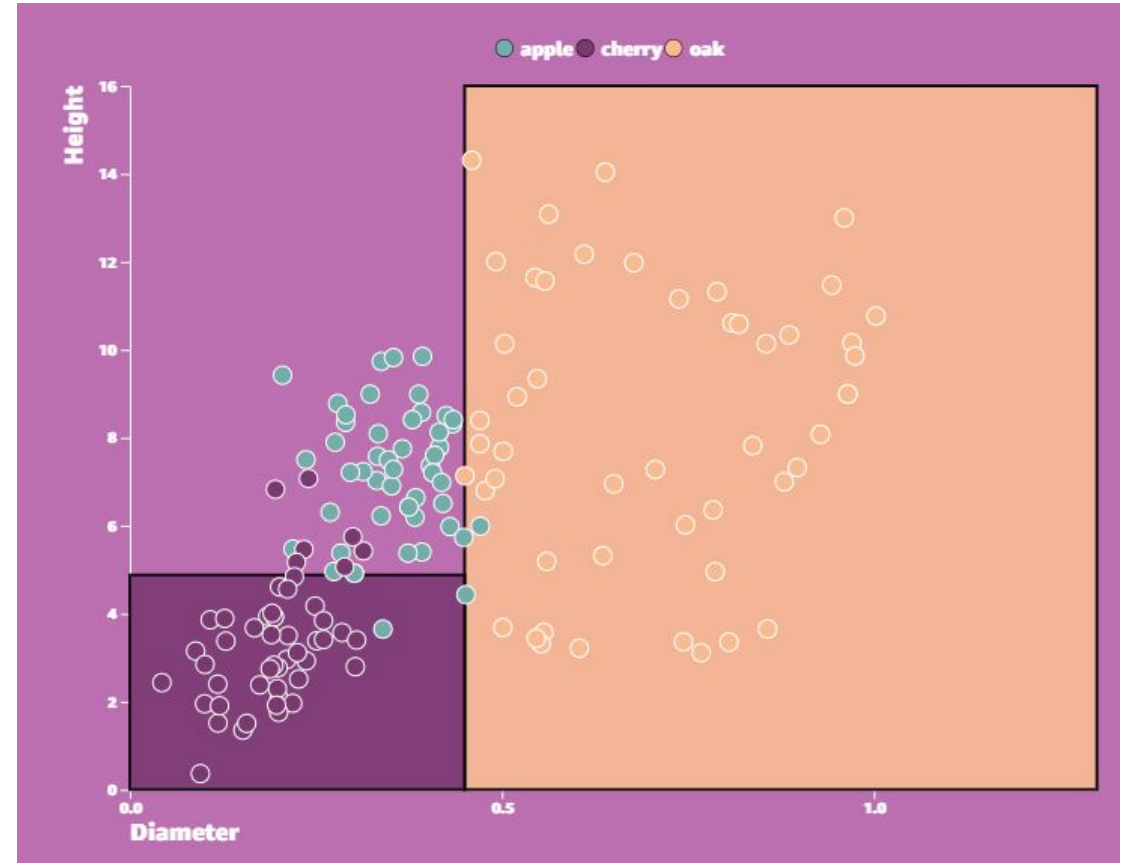
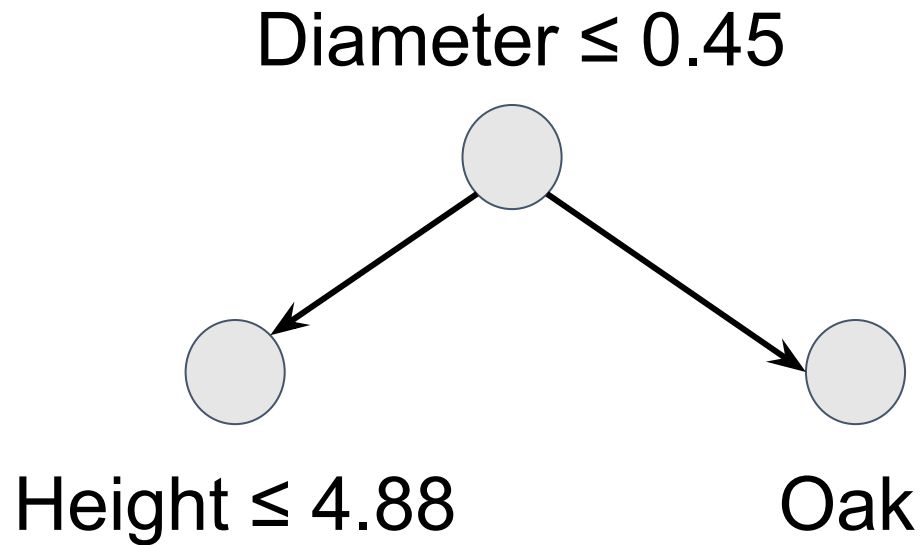
Example: Suppose a farmer want to determine whether a tree is an Apple, Cherry, or Oak tree given only it's diameter and height.



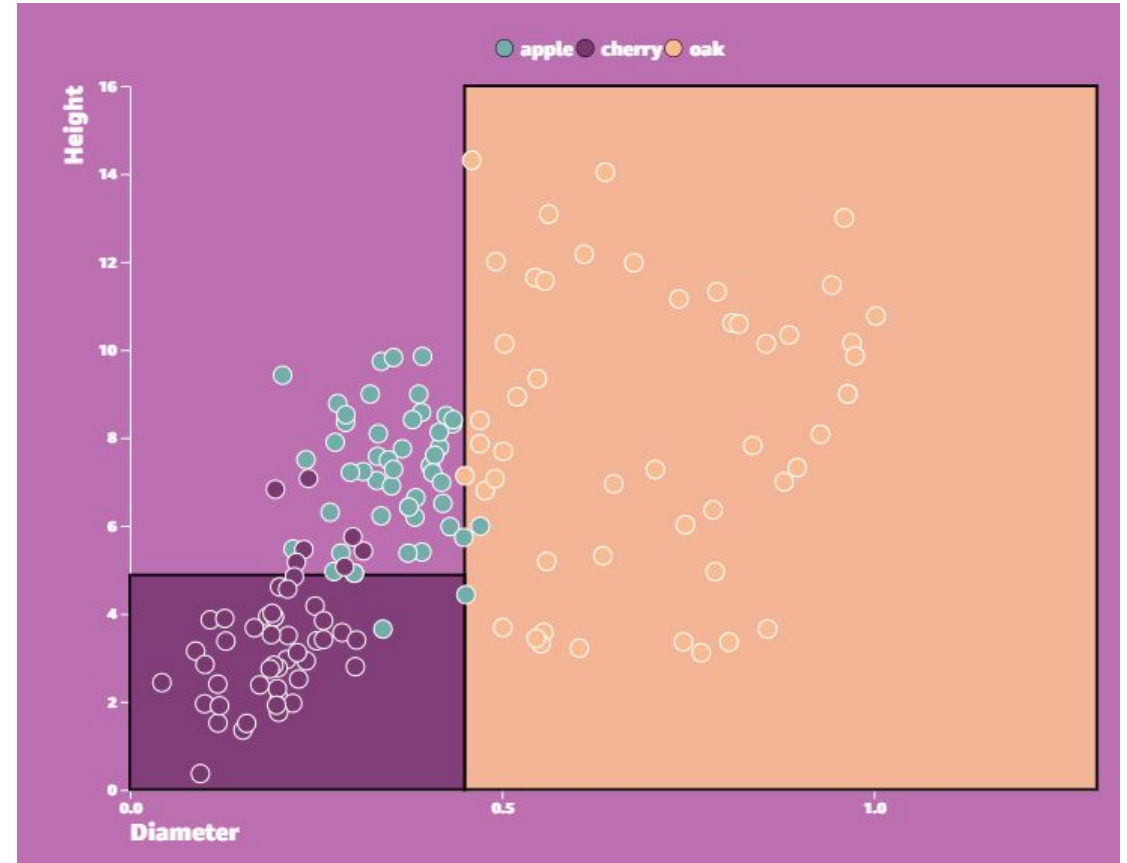
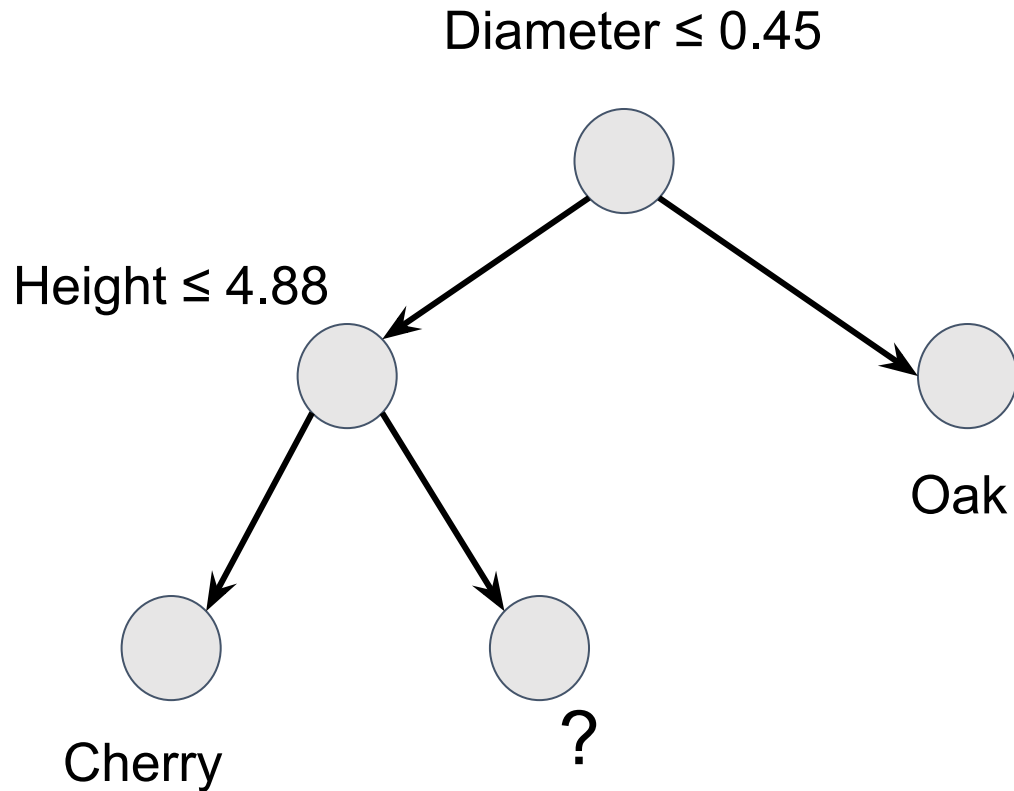
Classification Tree: Visual Example



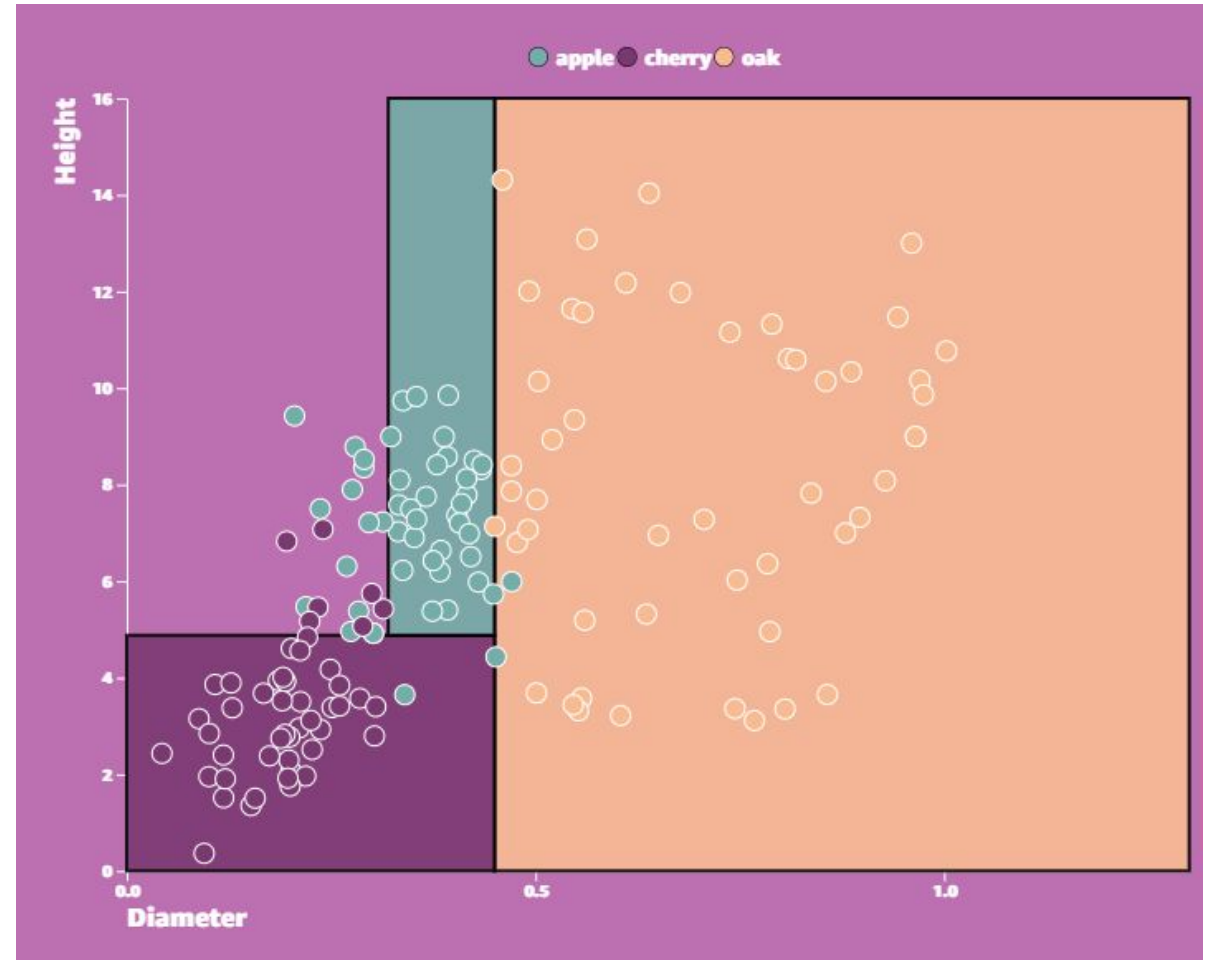
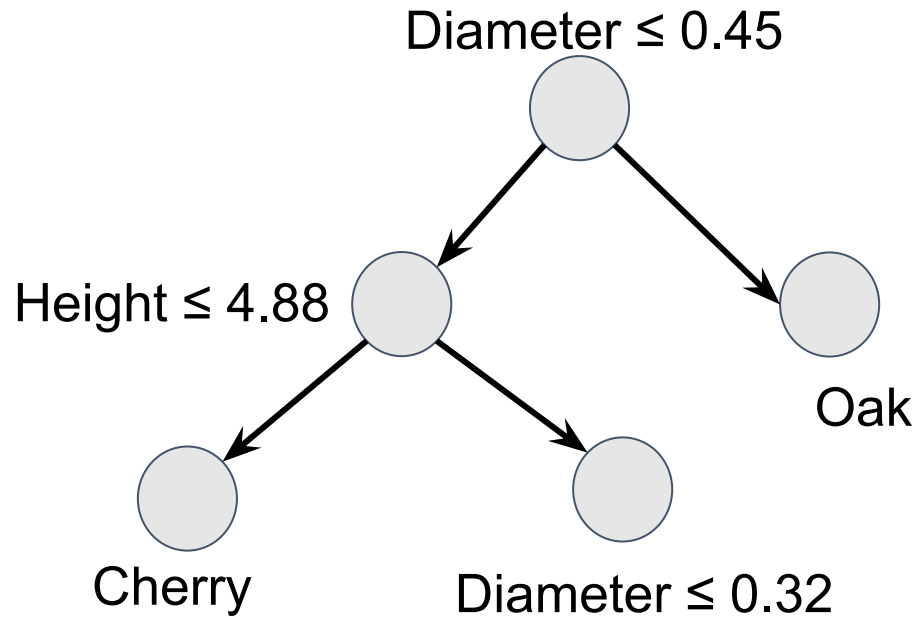
Classification Tree: Visual Example



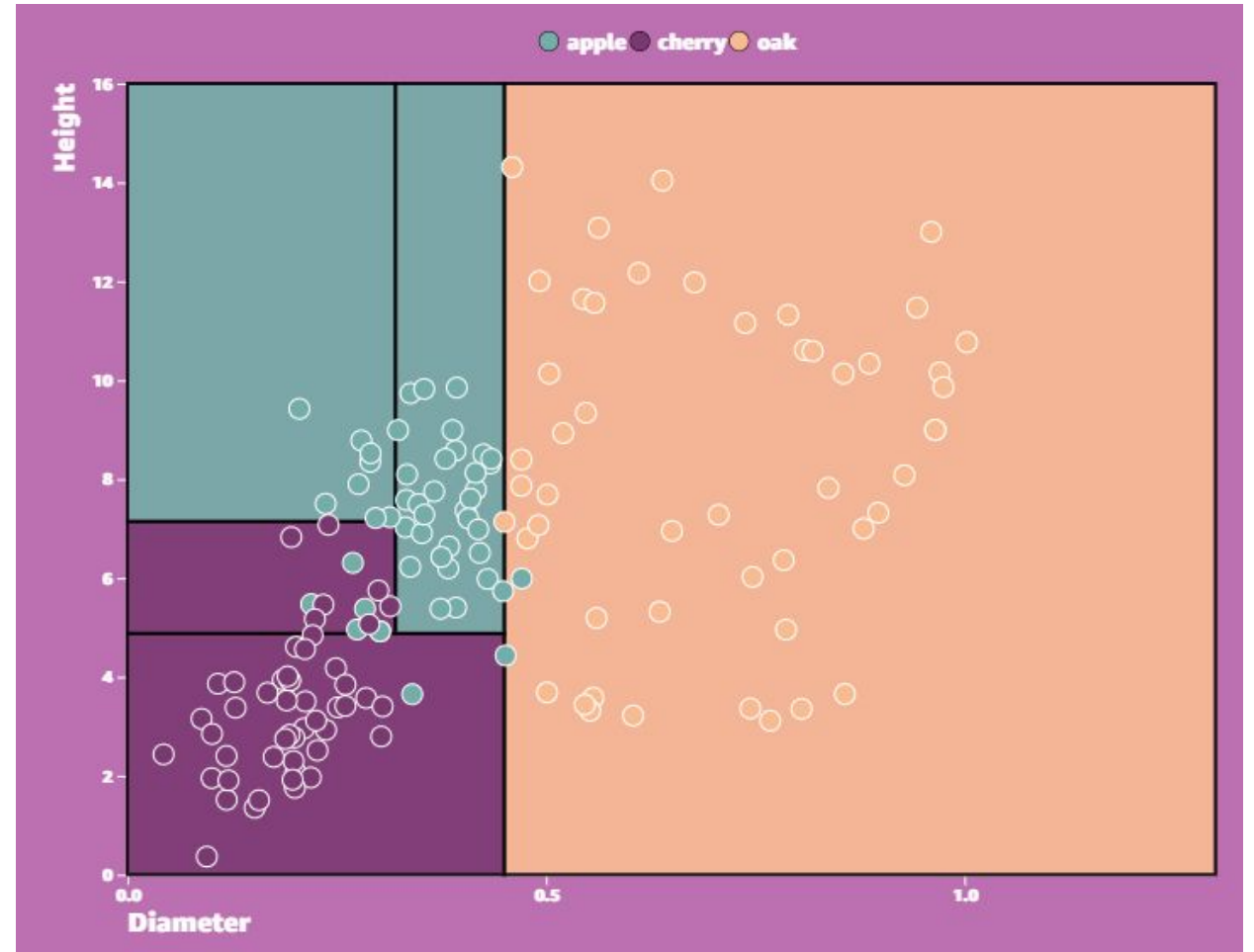
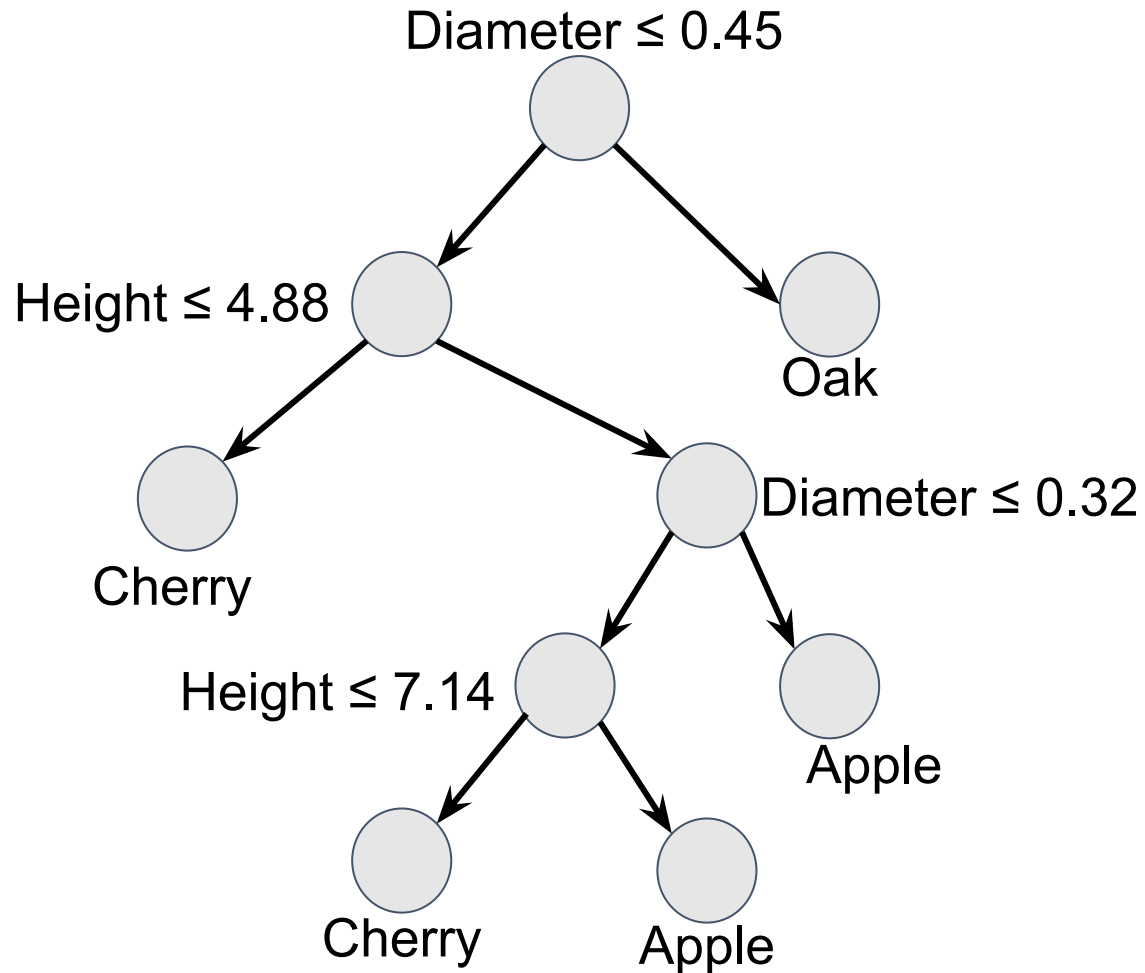
Classification Tree: Visual Example



Classification Tree: Visual Example



Classification Tree: Visual Example

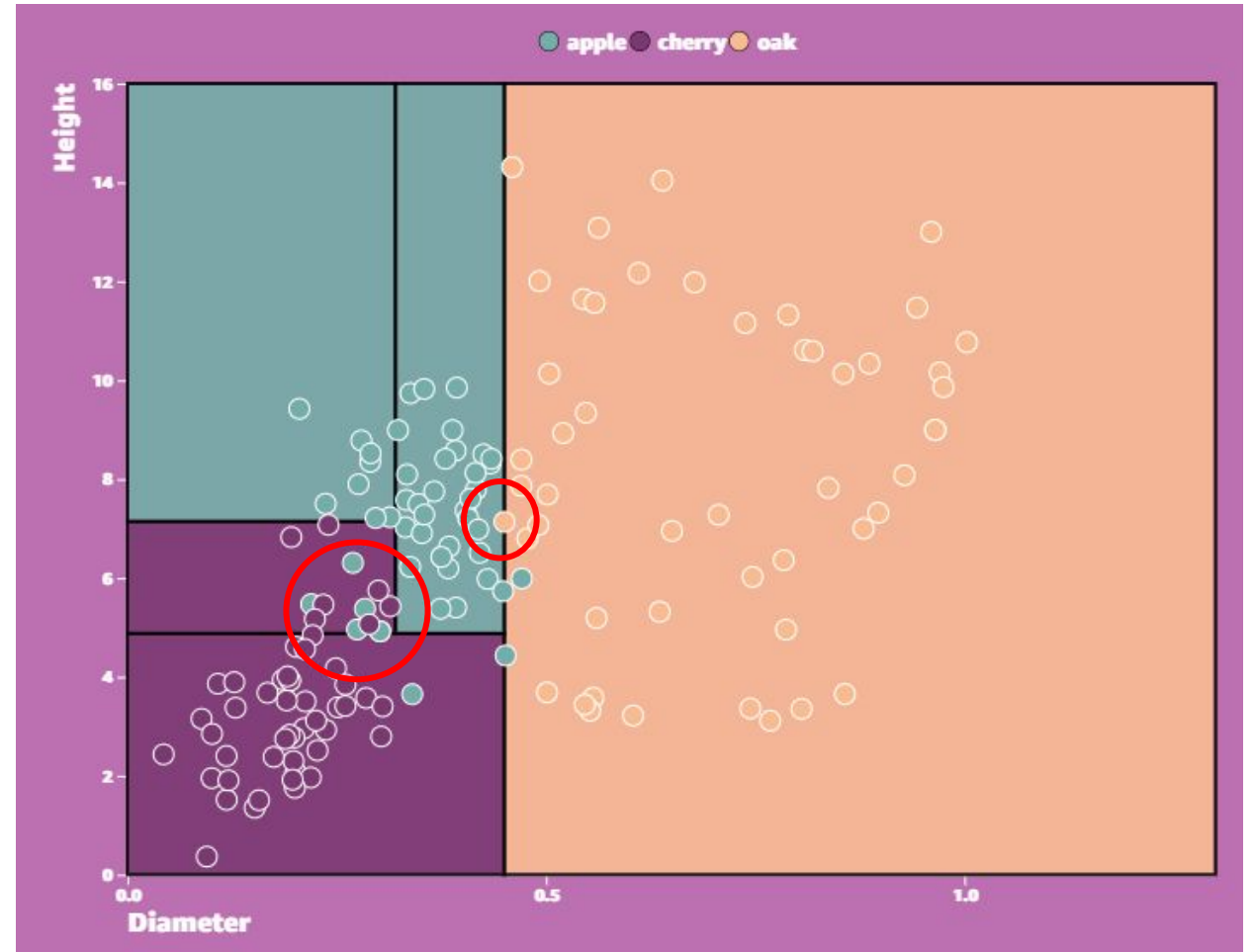


Classification Tree: Visual Example

What about the misclassified points?

We could create a deeper tree, but recall the bias-variance tradeoff.

If we make our tree too deep, it will overfit!



How to Construct a Regression Tree?

One common way to construct a regression tree using **recursive binary splitting**. It is a top-down, greedy approach.

- **Top-down** because it begins at the top of the tree (at which point all observations belong to a single region) and then successively splits the data; each split is indicated via two new branches further down on the tree.
- **Greedy** because at each step of the tree-building process, the best split is made at that particular step, rather than looking ahead and picking a split that will lead to a better tree in some future step.

We want to choose splits that minimize prediction error (**variance / SSE**).

Recursive Binary Splitting (Simple)

1. We begin at the root node containing all the data.
2. **Find** the best split by choosing the feature and split point that achieve the lowest SSE.
3. Using the **best split**, divide the dataset into **2 child nodes**.
4. Repeat steps 2-3 for each new node until a **stopping criterion** is reached.
5. Stop splitting and **assign predictions**: each leaf node gets a predicted value equal to the **mean of all target values** in that node.

Recursive Binary Splitting (Technical)

1. We begin at the root node containing all the data.
2. **Find** the best split by choosing the feature (j) and split point (s) that have the **lowest SSE**. For each feature, divide the data into 2 regions (R_1, R_2):

$$R_1(j, s) = \{x | x_j \leq s\} \qquad R_2(j, s) = \{x | x_j > s\}$$

$$SSE(j, s) = \sum_{x_i \in R_1(j, s)} (y_i - \bar{y}_{R_1})^2 + \sum_{x_i \in R_2(j, s)} (y_i - \bar{y}_{R_2})^2$$

3. Use the **best split** to divide the dataset into **2 child nodes**.
4. Repeat steps 2-3 for each new node until a **stopping criterion** is reached.
Example of stopping criterion: min # of samples in a node (e.g., ≤ 5), min reduction in SSE (no significant improvement), max tree depth.
5. Stop splitting and **assign predictions**: each leaf node gets a predicted value equal to the **mean of all target values** in that node.

How to Construct a Classification Tree?

We also use recursive binary splitting, but we cannot use SSE as a criterion for making the best split. Instead we aim to minimize **impurity**.

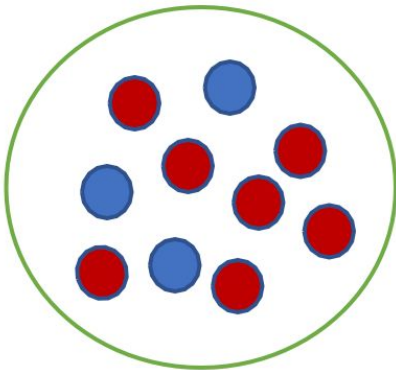
Regression tree: Best split = smallest **SSE**

Classification tree: Best split = smallest **impurity**

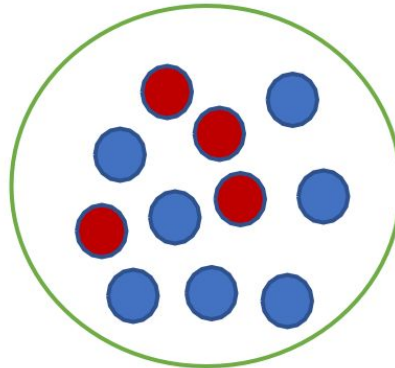
What is Impurity?

Impurity measures **how mixed** the classes are within a node of a decision tree. A node containing **multiple classes is impure** whereas a node including **only one class is pure**.

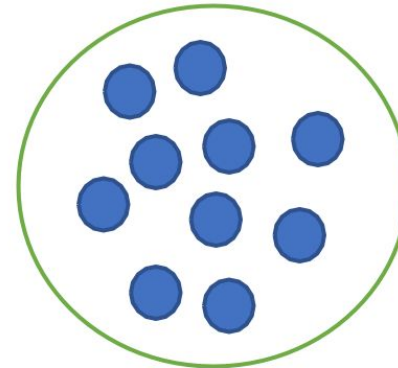
Very Impure



Less Impure



Pure



How do we Measure Impurity?

There are several ways to **measure impurity**. We will cover 2 methods:

1. Gini Impurity
2. Entropy

Once we have calculated impurity, the best split is the one with the **largest reduction in impurity**.

Gini Impurity

Gini impurity measures how often a randomly chosen element of a set would be misclassified if it were labeled randomly and independently according to the distribution of labels in the set.

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$$

K = the # of classes

$\hat{p}_{mk}$ = the proportion of training observations in the m th region that are from the k th class

The **lower the Gini Impurity, the better!** When Gini Impurity = 0, the node is completely pure. We choose the split that minimizes the Gini impurity.

Gini Impurity Example

Suppose we classifying Cats vs. Dogs and at a given node we have subset of our data with the following proportions: 30 samples are labelled “Dog” and 10 are labelled “Cat”. So the proportion of “Dog” labels is 75% and “Cat” is 25%. Following the formula for Gini Impurity we get:

$$0.75 * (1 - 0.75) = 0.1875$$

$$0.25 * (1 - 0.25) = 0.1875$$

$$G = 0.1875 + 0.1875 = 0.375$$

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$$

So if we were to randomly label a sample from this node according to the observed class proportions, we’d expect to be incorrect 37.5% of the time.

Gini Impurity Example

Suppose now our nodes splits perfectly so our subset of samples looks like: 40 samples are labelled “Dog” and 0 are labelled “Cat”. So the proportion of “Dog” labels is 100% and “Cat” is 0%. Following the formula for Gini Impurity we get:

$$1.0 * (1 - 1.0) = 0.0$$

$$0.0 * (1 - 0.0) = 0.0$$

$$G = 0.0 + 0.0 = 0.0$$

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$$

So if we were to randomly label a sample from this node according to the observed class proportions, we’d expect to be incorrect 0.0% of the time.

Entropy

Entropy the amount of information of some variable or event. Used it to identify regions of similar/pure or dissimilar/impure elements.

$$D = - \sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}$$

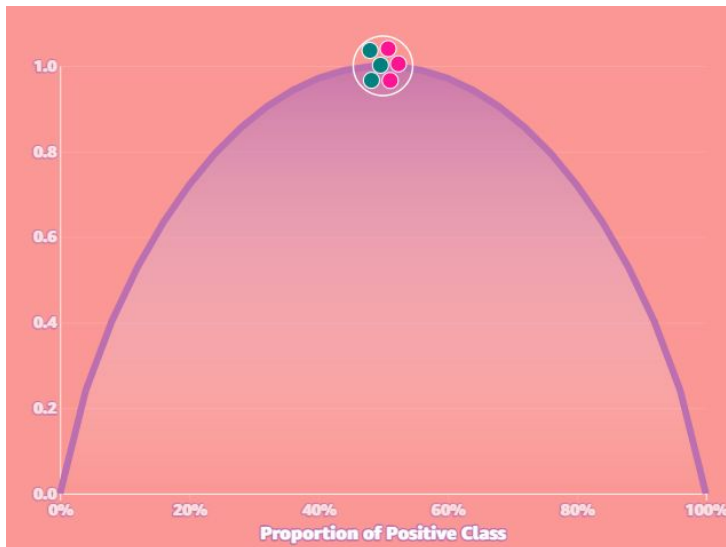
As with Gini Impurity, **lower entropy translates to higher purity.**

Entropy (Visual Example)

We use **entropy** to quantify the impurity of a collection of labeled data points. A node containing **multiple classes is impure** whereas a node including **only one class is pure**.

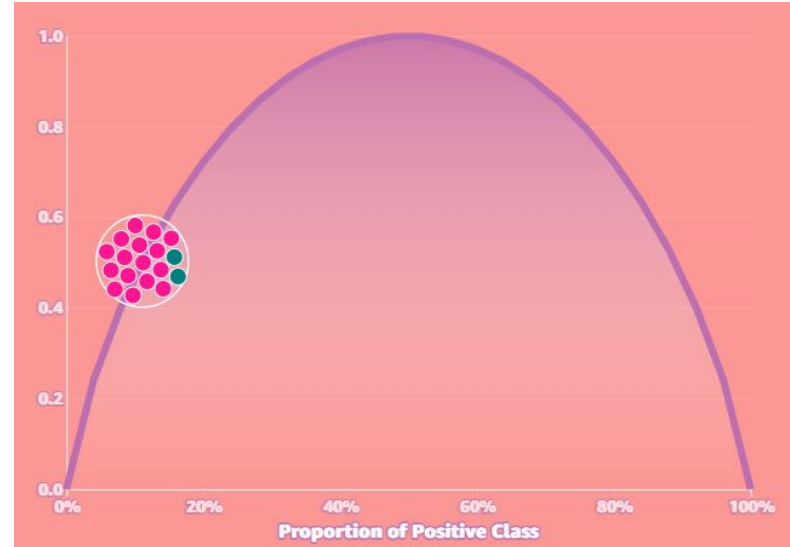
Entropy = 1

Positive = 3, Negative = 3



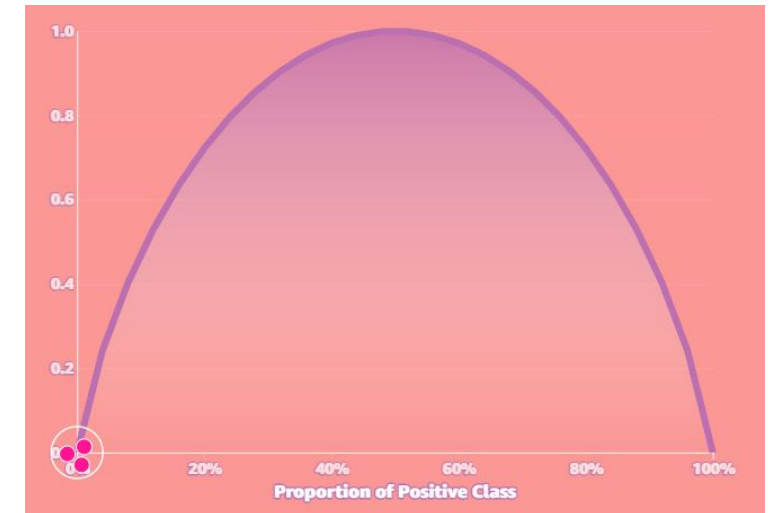
Entropy = 0.5

Positive = 2, Negative = 16



Entropy = 0

Positive = 0, Negative = 3



← More Impure

More Pure →

Information Gain

We **select the split** that yields the **largest reduction in entropy**, or equivalently, the **largest increase in information** where information gain is a measure of a reduction in uncertainty.

Higher Information Gain → better split (more reduction in uncertainty)

Decision Trees

Pros:

- Easy to understand and visualize.
- Requires little data preparation.
- Able to handle both numerical and categorical data.
- Don't rely heavily on strict assumptions about the data (e.g. linearity, normality, or equal variances).

Cons:

- Prone to overfitting (overly-complex trees).
- Can be unstable since small variations in the data might result in a completely different tree being generated.
- Not good at extrapolation since the predictions of DTs are neither smooth nor continuous.

Trees are High Variance

- Can grow too deep and become **overly complex** (overfit).
 - We can prevent this by:
 - Constraining their maximum depth
 - Limiting the number of leaves that can be created
 - Setting a minimum size for the amount of items in each leaf
- DTs are subject to **high variance**: depending on where the first split starts, the resulting tree can be very different. This makes trees high variance - because predictions can be wildly different even from data from the same dataset.

How Can We Reduce Variance?

This problem is mitigated by using decision trees within an **ensemble**.

We will cover 2 widely used ensemble methods:

- Bagging
- Boosting

Ensemble Learning

Ensemble learning is a machine learning technique that aggregates two or more learners (e.g. regression models, neural networks) to produce **better predictions**.

An **ensemble model** combines **several individual models** to produce more accurate predictions than a single model alone.

We will cover 2 common ensemble methods: **bagging and boosting**.

Bagging (Bootstrap Aggregating)

Bagging is an ensemble method that aims to reduce variance.

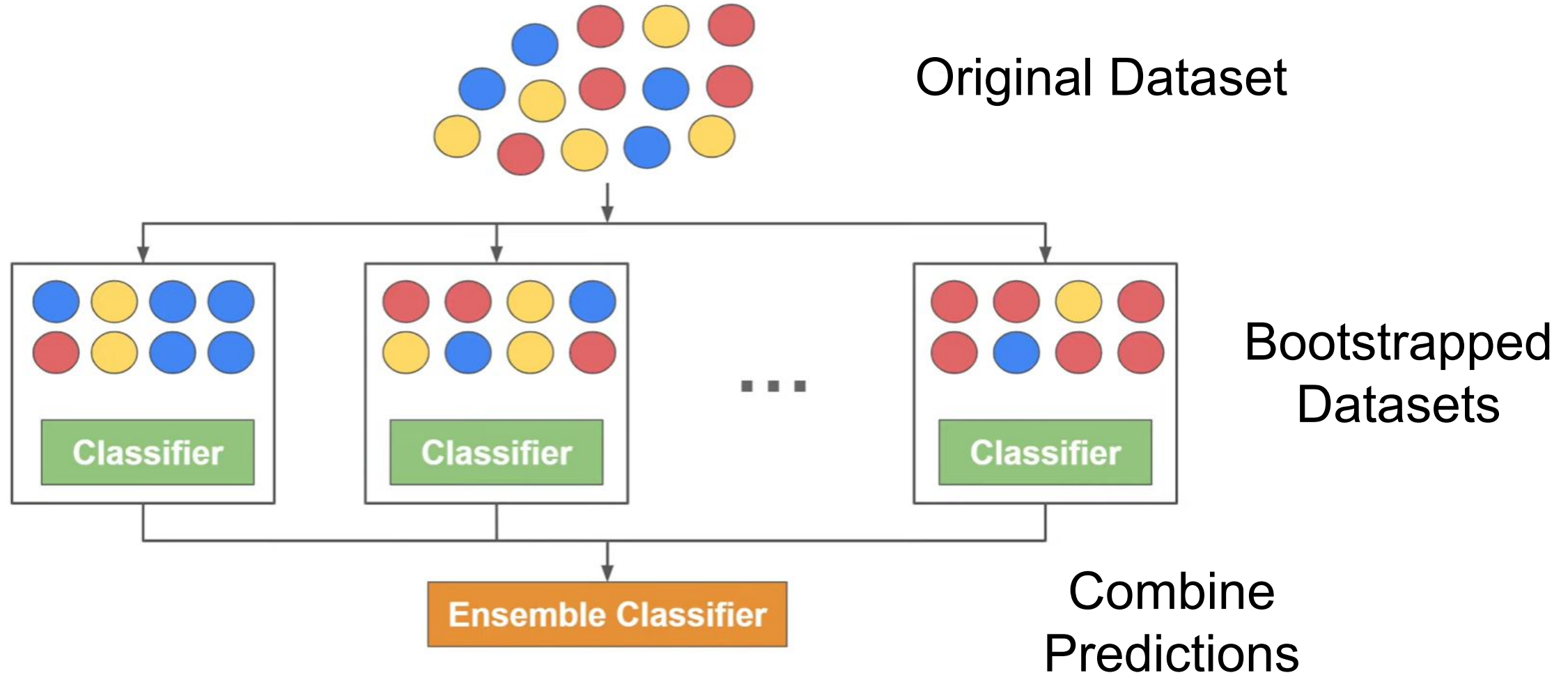
- Involves training **multiple independent models** on **different subsets of the dataset** and then **combining their predictions** to produce a final output.
- By combining predictions from models trained on **slightly different datasets**, the **overall variance of the model decreases**.

How Does Bagging Work?

Bagging works by **averaging over trees** trained on **bootstrapped samples** of our training data.

1. Create B bootstrap datasets by randomly sampling from the original dataset **with replacement**.
2. Train a separate model on **each subset**.
3. Determine the outcome by combining the predictions from all the models:
 - For classification → outcome is the **majority vote**
 - For regression → **average** the all the models' predictions

Bagging Visualization



Bagging Pros and Cons

Pros:

- Reduces variance.
- Improves stability (less likely to be influenced by outliers / noise).
- Can parallelize since models are independent.

Cons:

- Doesn't reduce bias.
- Bagged trees can look very similar (i.e. they are correlated). The correlation limits the benefit of averaging.
- Interpretation is lost / more difficult.

Random Forest (RF)

Random Forest (RF) is a special case of **bagging** where each tree only considers a random subset of features (not all features).

- By choosing random subsets of features, we **reduce correlation between trees** which leads to **lower variance**.

Why Random Forest (RF)?

One limitation of bagging is that bagged trees can look very similar (i.e. they are correlated). The correlation limits the benefit of averaging.

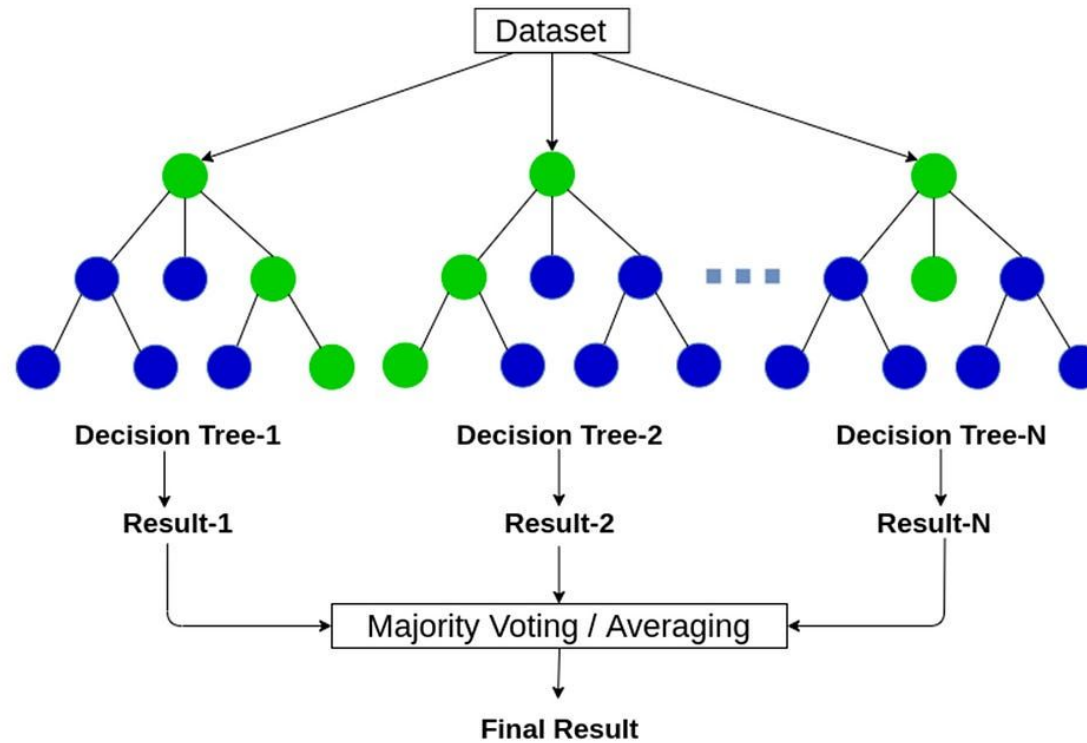
- **Random Forest** helps this by adding randomness.
 - At each split, instead of considering all features, the algorithm randomly selects a subset of features.
- This forces trees to explore different predictors, even if some are very strong, making them less correlated.

How Does RF Work?

Very similar to **bagging**, with one key difference:

1. Create B bootstrap datasets by randomly sampling from the original dataset **with replacement**.
2. Train a decision tree for each bootstrap dataset, using a **random subset of features at each step in the tree**.
3. Determine the outcome by combining the predictions from all the models:
 - For classification → outcome is the **majority vote**
 - For regression → **average** the all the models' predictions

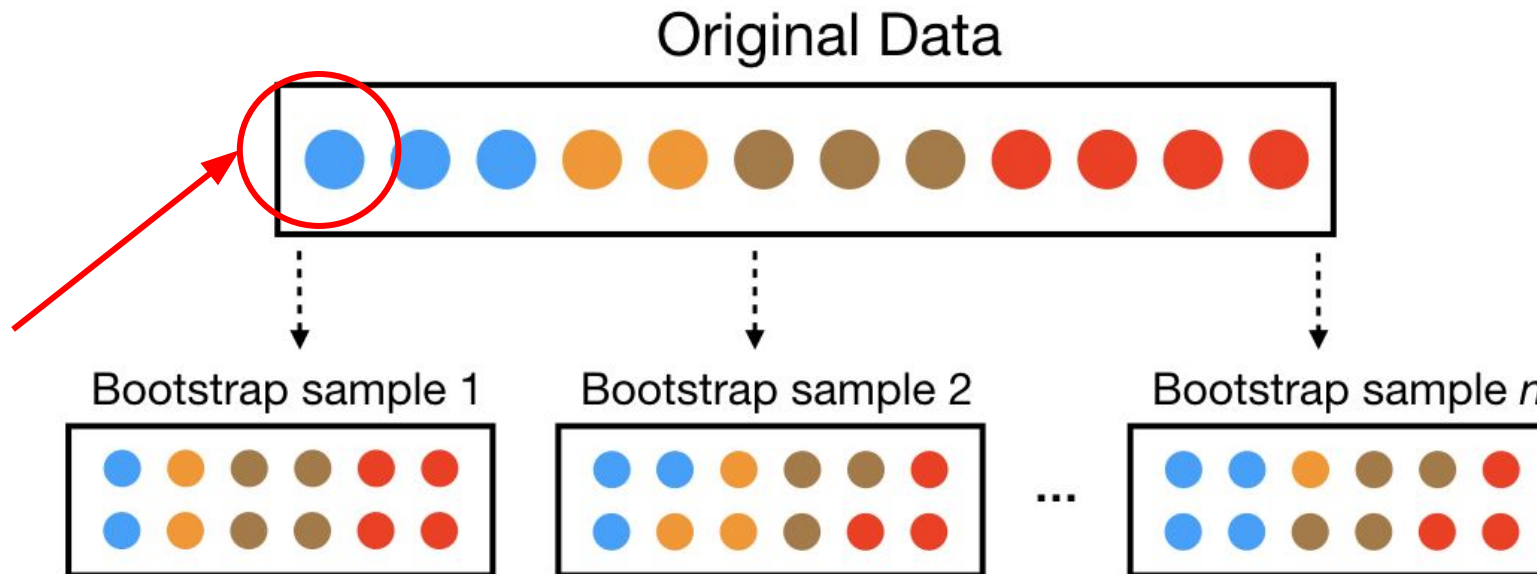
Random Forest Visualization



Out-of-Bag (OOB) Samples

Each tree is trained on a **bootstrap sample** (a random sample **with replacement**). On average, about one-third of the data is **not included** in that tree's bootstrap sample - these are called the **out-of-bag (OOB) samples**.

E.g. For Bootstrap Sample 1, the circled blue sample would be an out-of-bag sample.



Out-of-Bag (OOB) Error

We can measure the accuracy of our random forest is by calculating its performance on the **out-of-bag samples**.

For classification:

- The proportion of out-of-bag samples that were **incorrectly classified** are called the **out-of-bag (OOB) error**.

For regression:

- The **OOB error** is computed as the **mean squared error (MSE)** between the OOB predictions and the true target values.

Boosting

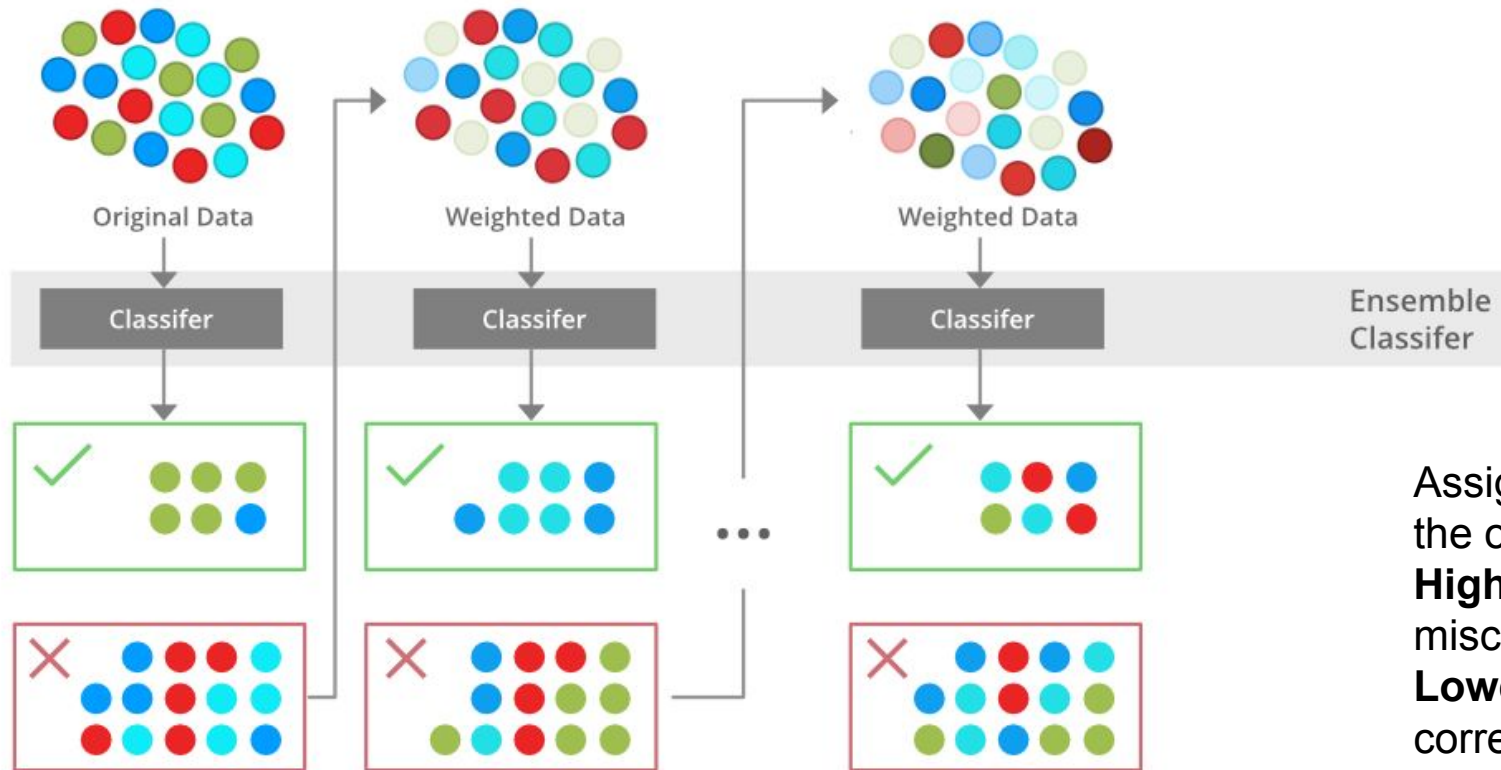
Boosting trains models sequentially, where each new model focuses on the errors of the previous ones.

- A series of models are trained and with each new model, the **weights** of the misclassified data in the previous model **are increased**.
- This **redistribution of weights** helps the algorithm identify the parameters that it needs to focus on to improve its performance.
- Individual models would not perform well on the entire dataset, but they work well for some part of the dataset. Essentially, by **combining several weak learners**, we end up with a **strong learner**.

Boosting Visualization

Initially, a model is built using the training data.

Subsequent models learn from the weighted dataset, focusing its attention on harder-to-learn examples (those with higher weights).



Assign weights to the data points in the original dataset:

Higher weights → points that were misclassified by the previous model

Lower weights → points that were correctly classified

Boosting Pros and Cons

Pros:

- Reduces **bias** by combining many weak models that correct each other's mistakes.

Cons:

- Sensitive to noisy data and outliers.
- Prone to overfitting.
- Harder to parallelize (because of sequential training).

AdaBoost (Adaptive Boosting)

AdaBoost (Adaptive Boosting) is one of the most classical (1995) forms of boosting.

1. Initialize every training sample to have the same importance (weight).
2. Train a simple decision tree.
3. Compute errors (misclassified samples).
4. Update sample weights.
 - Increase the weights of misclassified samples so they get more attention in the next round. **Correctly** classified samples get **lower weights**.
5. Repeat steps 2 to 4 (usually we specify the # of "boosting rounds") until a stopping criterion is reached.
6. Make the **final prediction**:
 - a. Classification → weighted vote
 - b. Regression → weighted average of predictions

**The weights in step 6 come from how well the model did in its own boosting round.*

Gradient Boosting

Instead of changing weights, **Gradient Boosting** builds each new model to **predict the residuals** (the errors) of the previous one.

1. Compute initial best guess (the mean as an example) for the target variable.
2. Compute the **pseudo-residuals**.
 - For regression: true value - predicted value. For classification: the negative gradient of the loss function.
3. Train a new tree to **predict the pseudo-residuals**.
4. **Update** our prediction by summing up the value from our first model with the predictions of all subsequent models.
5. Repeat steps 2 to 4 until a stopping criterion is reached.

XGBoost (eXtreme Gradient Boosting)

XGBoost is an optimized version of **Gradient Boosting** that uses **regularization** to prevent overfitting.

- Can handle missing values automatically during training.
- It is faster, efficient and supports handling both numerical and categorical variables (in versions > 1.5, see next slide).

A Note on Categorical Variables

Note that you must **encode categorical variables** before using the following models in sklearn:

- `sklearn.ensemble.AdaBoostClassifier` / `AdaBoostRegressor`
- `sklearn.ensemble.GradientBoostingClassifier` / `GradientBoostingRegressor`

For **XGBoost** versions newer than 1.5, you can use categorical variables if you set `enable_categorical=True`.

Feature Importance

We've seen that bagging and boosting improve prediction accuracy at the expense of interpretability.

Although it becomes harder to visualize a tree, we can get some information from **feature importance plots**.

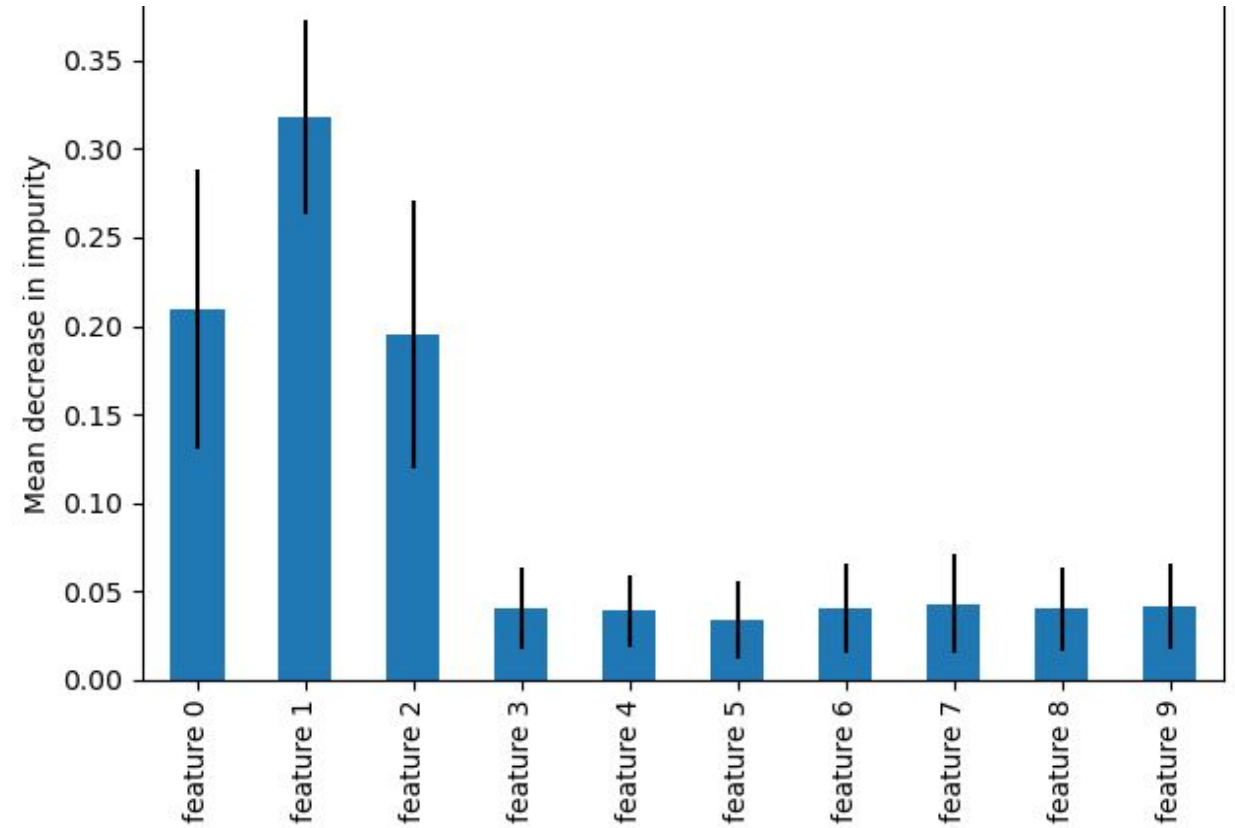
Feature importance plots shows how much each feature contributes to the model's predictions.

Feature Importance

1. Train your bagging or boosting model.
2. For each split in every tree, store how much the split reduced impurity.
3. For each feature, sum all the impurity reductions from splits that used that feature.
4. Sort the features by importance, and plot!

Feature Importance: Example Plot

In the feature importance plot below, we see that the model found the first 3 features the most important (they had the highest decrease in impurity).



Summary

- **Decision trees** are a machine learning model that produce an outcome by traversing if-then statements of a datasets features.
- They are high variance, so usually we use trees in an **ensemble**.
- **Bagging** trains multiple trees on bootstrapped datasets and combines their predictions.
 - Random Forest is an implementation of bagging.
- **Boosting** trains one tree sequentially, improving upon the errors each time.
 - AdaBoost, Gradient Boosting, and XGBoost are implementations of Boosting.
- We lose some interpretability when using ensemble methods, so we can use feature importance to understand our model better.

Further Resources

- [MLU-Explain Decision Trees](#)
- [StatQuest Random Forest](#)
- [Scikit-Learn Decision Trees](#)
- [How Gradient Boosting Works - RitvikMath](#)
- [StatQuest - Gradient Boost Visual Explanation](#)